UK Company Financial Health Review

UK-listed asset and wealth managers · data from audited group accounts · accessed 4-5 November 2026

1. The question

Which companies in one UK sector look financially stronger, and what should a lender or investor look into before relying on them? I chose six UK-listed asset and wealth managers: AJ Bell, Ashmore, Jupiter, Liontrust, Polar Capital and Premier Miton. These firms report their balance sheets in current and non-current items, so all five ratios can be worked out. Banks and insurers do not report this way, which is why I did not use them.

2. Data sources

Each company was identified by its Companies House number (04503206, 03675683, 06150195, 02954692, 04235369 and 06306664). The Companies House pages confirmed the company names, the year ends and, for most of them, the group accounts filed. The accounts PDFs held at Companies House could not be opened in this working environment. I therefore took the figures from the same audited group statements that each company publishes: its annual report, or its audited results announcement where the full report could not be read. Every figure in the workbook lists its source document and web link.

For each company I used two documents, each showing two years. That gives four years of figures: three years for the ratios, plus an earlier year as the starting point for revenue growth. Ashmore's results for the year to June 2026 have been published, but I could not retrieve the full statements. Its analysis therefore stops at FY2025.

3. Method

The figures were typed in exactly as printed. Amounts in £000 were then converted to £ millions. Blank fields were left blank rather than estimated. Every ratio in the workbook is a live Excel formula, and the same calculations are repeated in Python and checked with SQL queries.

- **Revenue growth** = this year's revenue ÷ last year's revenue − 1.
- **Profit margin** = profit before tax ÷ revenue. I used profit before tax because Polar Capital does not report an operating profit line, so it is the one profit measure all six share. Operating margin is shown alongside as a check.
- **Current ratio** = current assets ÷ current liabilities.
- **Debt-to-equity** = (borrowings + lease liabilities) ÷ total equity.
- **Interest coverage** = (profit before tax + finance costs) ÷ finance costs.

4. Findings

The table shows each company's latest year. The workbook sets out all three years.

Company	Year	Revenue £m	Growth	PBT margin	Current ratio	Debt / equity
AJ Bell	FY2025	317.8	+18.0%	43.4%	3.65x	0.06x
Polar Capital	FY2026	264.3	+19.0%	29.1%	2.53x	n/a
Jupiter	FY2025	465.7	+15.7%	28.3%	2.46x	0.04x
Ashmore	FY2025	144.4	−23.6%	75.2%	7.01x	0.01x
Liontrust	FY2026	134.4	−20.9%	10.7%	1.34x	0.03x
Premier Miton	FY2025	63.3	−1.1%	3.7%	1.27x	0.02x

Polar Capital's lease liabilities are not shown separately, so its debt-to-equity is left as n/a. Interest coverage is in the workbook. It is very high wherever the only interest cost is on office leases.

KPI screening

The five ratios, plus operating margin and the change in equity over three years, were turned into seven KPIs, each rated green, amber or red against fixed bands (for example, a profit margin of 25% or more is green, under 10% is red). Green scores 2 points, amber 1 and red 0, and KPIs with no data are left out. On this screen AJ Bell and Jupiter score 100% on all seven KPIs, and Polar Capital 100% on the five it reports. Ashmore scores 67%, green on its balance sheet but red on revenue growth and equity change. Liontrust

scores 43% and Premier Miton 17%. The bands are my own reading aids, and the score shows where to look first; it is not a rating.

Stronger on these measures

- **AJ Bell.** Revenue rose from £218.2m in FY2023 to £317.8m in FY2025, growing 33.2%, 23.5% and 18.0% a year. The profit margin edged up from 40.2% to 43.4%. The current ratio held at about 3.6x, and its only debt is £12.1m of leases against £217.5m of equity.
- **Polar Capital.** Revenue grew in each of the three years, by 6.7%, 13.9% and 19.0%, reaching £264.3m in FY2026. The margin stayed between 23% and 29%, and the current ratio was steady at about 2.5x. Its only interest cost is about £0.2m on leases.
- **Jupiter.** The FY2023 margin was only 2.3% because of a £76.2m goodwill write-down. It then recovered to 21.9% and 28.3%. Jupiter repaid its £50m subordinated notes in April 2025, so debt-to-equity fell from 0.12x to 0.04x. The current ratio improved from 2.09x to 2.46x.

Mixed

- **Ashmore.** Ashmore has the largest liquidity cushion (current ratio 7.0x to 10.7x) and no debt. However, revenue fell from £193.2m to £144.4m over the three years, and equity fell from £913.0m to £790.8m. Its 75.2% margin is flattered by £51.1m of finance income, which is 47% of profit before tax. Operating margin, at 30% to 40%, gives a better view of the fee business.

Weaker on these measures

- **Liontrust.** Revenue fell in each of the three years, from £243.3m in FY2023 to £134.4m in FY2026, a drop of 45%. It made a small loss before tax in FY2024. Equity fell from £221.0m to £107.9m. Its current ratio (1.34x) is the second lowest in the group.
- **Premier Miton.** Revenue fell from £90.2m in FY2022 to £63.3m in FY2025, about 30% lower. The margin narrowed each year, from 7.9% to 4.9% and then 3.7%. It has no bank debt, but has the lowest current ratio at 1.27x.

5. Risks a lender or investor should examine

- **Market and outflow risk.** Fees are charged on assets under management, so a market fall or client outflows reduce revenue quickly. This shows clearly at Liontrust, Premier Miton and Ashmore. Ask for the latest flows and assets under management, not just the accounts.
- **Quality of profit.** Ashmore's profit depends heavily on returns from its own investments, and Jupiter's FY2023 result was hit by a one-off write-down. Check how much profit comes from the core fee business.
- **Interest-rate sensitivity.** For AJ Bell and other platforms, part of revenue is interest earned on client cash. A cut in UK interest rates could reduce it. Ask what share of revenue this is.
- **What the current ratio contains.** Money moving between investors and funds can sit in receivables and payables. These amounts are large and largely offset each other, which pulls the current ratio towards 1x. Ask for the breakdown before treating 1.3x as a warning sign.
- **Shrinking equity and capital needs.** Liontrust's equity halved in three years as dividends and buybacks ran ahead of profit. Check dividend cover and the regulatory capital surplus reported in each firm's Pillar 3 or ICARA disclosures.
- **Hidden obligations.** Most of these firms owe little apart from leases, so debt ratios say little about them. A lender should look at covenants, guarantees, seed-capital commitments and any defined benefit pension scheme.

6. Limits of the data

- **Different dates.** The year ends are March, June, September and December, so the latest years run from June 2025 to March 2026. Market conditions were not the same across these dates.
- **Source route.** The figures come from the audited statements each company published, not from Companies House PDFs, which could not be read here. AJ Bell's figures come from its audited results announcements. The Companies House pages for AJ Bell and Ashmore came from older cached copies, and Jupiter's page could not be opened.

- **Missing figures.** Polar Capital does not report an operating profit line or separate lease liabilities. Ashmore reports no finance cost line. These cells are marked n/a, not estimated.
- **Reporting differences.** Each firm labels its lines differently ('net revenue', 'gross profit', 'net income'). Some prior years were re-presented. Each case is listed in the workbook's Data_Log sheet.
- **Private companies.** These are listed groups, but the same approach applied to private companies is harder. Small companies can file abridged accounts with no profit and loss account, so growth and margin cannot be calculated at all.

7. Conclusion

On these five measures, AJ Bell, Polar Capital and Jupiter look the strongest. They show growing revenue, healthy margins, comfortable current ratios and little or no borrowing. Ashmore has a very strong balance sheet but shrinking fee income. Liontrust and Premier Miton show falling revenue and thin margins, and their questions are about the trend more than about solvency today. None of this proves that any firm is safe or likely to fail. The ratios show where a lender or investor should ask further questions before deciding.

8. Sources and project files

All links were checked on 5 November 2026. Figures were accessed on 4-5 November 2026.

S1 AJ Bell plc: FY2025 results announcement (audited group statements). Open source

S2 AJ Bell plc: FY2023 results announcement (audited group statements). Open source

S3 Ashmore Group plc: Final results for the year ended 30 June 2025 (RNS 5 Sep 2025). Open source

S4 Ashmore Group plc: Final results for the year ended 30 June 2023 (RNS 6 Sep 2023). Open source

S5 Jupiter Fund Management plc: Annual Report and Accounts 2025 (ESEF filing). Open source

S6 Jupiter Fund Management plc: Annual Report and Accounts 2023. Open source

S7 Liontrust Asset Management plc: Annual Report and Financial Statements 2026. Open source

S8 Liontrust Asset Management plc: Annual Report and Financial Statements 2024 (ESEF filing). Open source

S9 Polar Capital Holdings plc: Group audited results for the year ended 31 March 2026 (RNS 1 Jul 2026). Open source

S10 Polar Capital Holdings plc: Final results for the year ended 31 March 2024. Open source

S11 Premier Miton Group plc: Full year results to 30 September 2025 (RNS 4 Dec 2025). Open source

S12 Premier Miton Group plc: Full year results to 30 September 2023 (RNS 5 Dec 2023). Open source

Companies House pages: AJ Bell (04503206) · Ashmore (03675683) · Jupiter (06150195) · Liontrust (02954692) · Polar Capital (04235369) · Premier Miton (06306664)

Project repository (workbook, data, code and dashboard images): https://github.com/Mumeen-Gbolagade/UK-Financial-Firms-Health-Review-

Direct files: Excel workbook · Dashboard image · KPI scorecard · SQL queries · Python code

9. Tools used

Tool	What it was used for
Companies House (find-and-update service)	Confirming company names and numbers, SIC codes, year ends and filed accounts records.
Company annual reports and audited results	Source of every financial figure (see the Sources sheet for links and access dates).
Python 3 with pandas	Importing the raw figures, converting units, quality checks, ratio and KPI calculations (src/analysis.py).
SQL (SQLite via Python's sqlite3)	Checks on the results: latest-year snapshot, three-year averages, revenue declines, thin margins, missing inputs (sql/queries.sql).
Microsoft Excel format (openpyxl)	Workbook with live formulas, conditional formatting and native charts. The Dashboard sheet is the BI layer of the project.
matplotlib and Pillow	Summary dashboard image and trimmed workbook screenshots for the README (src/make_dashboard.py, src/make_screenshots.py).
GitHub	Hosting the project; links in the workbook and report point to the repository so they work after download (src/config.py, src/check_links.py).
python-docx and LibreOffice	Writing this report and converting it to PDF; LibreOffice also recalculated and checked every workbook formula.

BI platform	No separate BI tool (such as Power BI or Tableau) was used. outputs/ratios.csv and outputs/financials.db can be loaded into one directly.

Appendix: project code

The complete code used for the analysis, as stored in the project folder. Run the scripts in this order: analysis.py, build_workbook.py, make_dashboard.py, make_screenshots.py, build_report.py, then check_links.py. The KPI bands are in data/kpi_thresholds.csv. No API keys or passwords are included; the optional Companies House download reads a key from the CH_API_KEY environment variable.

A1. SQL checks (sql/queries.sql)

```
-- UK Company Financial Health Review: SQL checks
-- These run against the SQLite database built by src/analysis.py (outputs/financials.db).
-- Table "ratios" holds one row per company and financial year, in GBP millions.

-- name: latest_year_snapshot
-- The most recent financial year for each company, with its key ratios.
SELECT r.company, r.fy_label, r.revenue_m, r.revenue_growth, r.pbt_margin,
       r.current_ratio, r.debt_to_equity, r.interest_cover
FROM ratios r
JOIN (SELECT company, MAX(fy_end) AS last_end FROM ratios GROUP BY company) l
  ON r.company = l.company AND r.fy_end = l.last_end
ORDER BY r.pbt_margin DESC;

-- name: three_year_average
-- Average of each ratio over the three analysed years (base year excluded).
SELECT company,
       ROUND(AVG(revenue_growth), 4) AS avg_revenue_growth,
       ROUND(AVG(pbt_margin), 4)     AS avg_pbt_margin,
       ROUND(AVG(current_ratio), 2)  AS avg_current_ratio,
       ROUND(MIN(current_ratio), 2)  AS lowest_current_ratio
FROM ratios
WHERE in_analysis = 1
GROUP BY company
ORDER BY avg_pbt_margin DESC;

-- name: revenue_declines
-- Every analysed year where revenue fell.
SELECT company, fy_label, revenue_m, ROUND(revenue_growth, 4) AS revenue_growth
FROM ratios
WHERE in_analysis = 1 AND revenue_growth < 0
ORDER BY revenue_growth;

-- name: loss_or_thin_margin_years
-- Years with a loss before tax or a profit before tax margin under 5%.
SELECT company, fy_label, profit_before_tax_m, ROUND(pbt_margin, 4) AS pbt_margin
FROM ratios
WHERE in_analysis = 1 AND pbt_margin < 0.05
ORDER BY pbt_margin;

-- name: missing_inputs
-- Count of ratio results that could not be calculated, by company.
SELECT company,
       SUM(CASE WHEN debt_to_equity IS NULL THEN 1 ELSE 0 END) AS missing_debt_to_equity,
       SUM(CASE WHEN interest_cover IS NULL THEN 1 ELSE 0 END) AS missing_interest_cover
FROM ratios
WHERE in_analysis = 1
GROUP BY company;
```

A2. Python: import, cleaning, quality checks, ratios and SQL (src/analysis.py)

```
"""
UK Company Financial Health Review - data import, cleaning and ratio analysis.

Steps
  1. (Optional) pull company profiles and accounts filing lists from the
     Companies House API. Needs your own free API key in the CH_API_KEY
     environment variable. No key is stored in this project.
  2. Load the figures extracted from each company's audited accounts
     (data/raw_financials.csv) and the company register (data/companies.csv).
  3. Check and clean: convert everything to GBP millions, log missing values
     and unusual figures. Nothing is filled in or estimated.
  4. Calculate revenue growth, profit margin, current ratio, debt-to-equity
     and interest coverage.
  5. Load the results into SQLite and run the checks in sql/queries.sql.

Run from the project folder:
    python src/analysis.py
"""

from __future__ import annotations

import json
import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
OUT = ROOT / "outputs"
SQL_FILE = ROOT / "sql" / "queries.sql"

MONEY_COLS = [
    "revenue", "net_revenue", "operating_profit", "finance_income",
    "finance_costs", "profit_before_tax", "profit_for_year",
```

```
    "current_assets", "current_liabilities", "borrowings",
    "lease_liabilities", "total_equity",
]
UNIT_FACTOR = {"GBP000": 1 / 1000, "GBPm": 1.0}


# ---------------------------------------------------------------- step 1
def fetch_companies_house(numbers: list[str]) -> None:
    """Save each company's profile and accounts filing list as JSON.

    Skipped quietly when CH_API_KEY is not set. The key is read from the
    environment only and is never written to disk.
    """
    key = os.environ.get("CH_API_KEY")
    if not key:
        print("Step 1: CH_API_KEY not set, skipping the Companies House download.")
        return
    import requests  # imported here so the rest runs without it

    folder = DATA / "companies_house"
    folder.mkdir(exist_ok=True)
    base = "https://api.company-information.service.gov.uk"
    for number in numbers:
        profile = requests.get(f"{base}/company/{number}", auth=(key, ""), timeout=30)
        filings = requests.get(
            f"{base}/company/{number}/filing-history",
            params={"category": "accounts", "items_per_page": 20},
            auth=(key, ""), timeout=30,
        )
        for name, resp in (("profile", profile), ("accounts_filings", filings)):
            if resp.ok:
                (folder / f"{number}_{name}.json").write_text(json.dumps(resp.json(), indent=2))
            else:
                print(f"  {number} {name}: HTTP {resp.status_code}")
    print(f"Step 1: Companies House data saved to {folder}")


# ---------------------------------------------------------------- step 2-3
def load_and_clean() -> tuple[pd.DataFrame, pd.DataFrame]:
    raw = pd.read_csv(DATA / "raw_financials.csv", dtype={"company_number": str})
    companies = pd.read_csv(DATA / "companies.csv", dtype={"company_number": str})

    unknown_units = set(raw["units"]) - set(UNIT_FACTOR)
    if unknown_units:
        raise ValueError(f"Unexpected units: {unknown_units}")

    clean = raw.copy()
    factor = clean["units"].map(UNIT_FACTOR)
    for col in MONEY_COLS:
        clean[col + "_m"] = clean[col] * factor
    clean["fy_end"] = pd.to_datetime(clean["fy_end"])
    clean = clean.sort_values(["company", "fy_end"]).reset_index(drop=True)
    clean = clean.merge(companies[["company", "short_name"]], on="company", how="left")

    # Each company has four years; the earliest is only used as the base for growth.
    clean["year_rank"] = clean.groupby("company")["fy_end"].rank(ascending=False)
    clean["in_analysis"] = (clean["year_rank"] <= 3).astype(int)

    log = []
    for _, r in clean.iterrows():
        for col in MONEY_COLS:
            if pd.isna(r[col]):
                log.append((r["company"], r["fy_label"], "Missing value", col,
                            "Not shown in the retrieved statements; left blank, not estimated."))
        if r["profit_before_tax"] < 0:
            log.append((r["company"], r["fy_label"], "Unusual figure", "profit_before_tax",
                        "Loss before tax."))
        if pd.notna(r["operating_profit"]) and r["operating_profit"] != 0:
            gap = (r["profit_before_tax"] - r["operating_profit"]) / abs(r["operating_profit"])
            if abs(gap) > 0.30:
                log.append((r["company"], r["fy_label"], "Unusual figure", "profit_before_tax",
                            f"Profit before tax differs from operating profit by {gap:+.0%} "
                            "(items below operating profit, e.g. finance income or impairment)."))
    quality = pd.DataFrame(log, columns=["company", "fy_label", "type", "field", "detail"])
    return clean, quality


# ---------------------------------------------------------------- step 4
def ratio_or_none(num: pd.Series, den: pd.Series) -> pd.Series:
    """Divide, returning None when either side is missing or the divisor is zero."""
    out = num / den
    out[(den == 0) | den.isna() | num.isna()] = None
    return out


def calculate_ratios(clean: pd.DataFrame) -> pd.DataFrame:
    df = clean.copy()
    prev_rev = df.groupby("company")["revenue_m"].shift(1)
    df["revenue_growth"] = ratio_or_none(df["revenue_m"], prev_rev) - 1
    df["pbt_margin"] = ratio_or_none(df["profit_before_tax_m"], df["revenue_m"])
    df["operating_margin"] = ratio_or_none(df["operating_profit_m"], df["revenue_m"])
    df["current_ratio"] = ratio_or_none(df["current_assets_m"], df["current_liabilities_m"])
    debt = df["borrowings_m"] + df["lease_liabilities_m"]  # NaN if leases missing
    df["total_debt_m"] = debt
    df["debt_to_equity"] = ratio_or_none(debt, df["total_equity_m"])
    ebit = df["profit_before_tax_m"] + df["finance_costs_m"]
    df["interest_cover"] = ratio_or_none(ebit, df["finance_costs_m"])
    return df
```

```
def kpi_scorecard(ratios: pd.DataFrame) -> pd.DataFrame:
    """Latest-year KPIs per company with a green / amber / red status.

    Bands come from data/kpi_thresholds.csv. They are reading aids chosen by
    the analyst, not industry rules, and the score is a screen, not a rating.
    """
    bands = pd.read_csv(DATA / "kpi_thresholds.csv")
    df = ratios.sort_values(["company", "fy_end"])
    latest = df.groupby("company").tail(1).set_index("company")
    base_equity = df.groupby("company")["total_equity_m"].first()   # base year, 3 years earlier
    latest["equity_change_3y"] = latest["total_equity_m"] / base_equity - 1
    points = {"Green": 2, "Amber": 1, "Red": 0}
    rows = []
    for company, r in latest.iterrows():
        out = {"company": company, "short_name": r["short_name"], "fy_label": r["fy_label"]}
        got, possible = 0, 0
        for b in bands.itertuples():
            v = r[b.kpi]
            if pd.isna(v):
                status = "No data"
            elif b.better == "higher":
                status = "Green" if v >= b.green_from else "Amber" if v >= b.amber_from else "Red"
            else:
                status = "Green" if v <= b.green_from else "Amber" if v <= b.amber_from else "Red"
            out[b.kpi] = v
            out[b.kpi + "_status"] = status
            if status in points:
                got += points[status]
                possible += 2
        out["kpis_scored"] = possible // 2
        out["screening_score"] = got / possible if possible else None
        rows.append(out)
    return pd.DataFrame(rows).sort_values("screening_score", ascending=False)


# ---------------------------------------------------------------- step 5
def run_sql(ratios: pd.DataFrame) -> dict[str, pd.DataFrame]:
    db_path = OUT / "financials.db"
    if db_path.exists():
        db_path.unlink()
    cols = ["company", "short_name", "fy_label", "fy_end", "in_analysis", "revenue_m",
            "profit_before_tax_m", "revenue_growth", "pbt_margin", "operating_margin",
            "current_ratio", "debt_to_equity", "interest_cover"]
    table = ratios[cols].copy()
    table["fy_end"] = table["fy_end"].dt.strftime("%Y-%m-%d")
    with sqlite3.connect(db_path) as con:
        table.to_sql("ratios", con, index=False)
        results = {}
        for block in SQL_FILE.read_text().split("-- name: ")[1:]:
            name, _, body = block.partition("\n")
            results[name.strip()] = pd.read_sql_query(body, con)
    return results


def main() -> None:
    OUT.mkdir(exist_ok=True)
    companies = pd.read_csv(DATA / "companies.csv", dtype={"company_number": str})
    fetch_companies_house(companies["company_number"].tolist())

    clean, quality = load_and_clean()
    print(f"Step 2-3: {len(clean)} company-years loaded, {len(quality)} data notes logged.")

    ratios = calculate_ratios(clean)
    ratios.to_csv(OUT / "ratios.csv", index=False, float_format="%.4f")
    quality.to_csv(OUT / "data_quality_log.csv", index=False)
    print("Step 4: ratios written to outputs/ratios.csv")
    kpis = kpi_scorecard(ratios)
    kpis.to_csv(OUT / "kpi_scorecard.csv", index=False, float_format="%.4f")
    print("        KPI scorecard written to outputs/kpi_scorecard.csv")
    print(kpis[["short_name", "fy_label", "kpis_scored", "screening_score"]].to_string(index=False))

    results = run_sql(ratios)
    with pd.ExcelWriter(OUT / "sql_results.xlsx") as xw:
        for name, frame in results.items():
            frame.to_excel(xw, sheet_name=name[:31], index=False)
    print("Step 5: SQL checks written to outputs/sql_results.xlsx")
    print("\nLatest year snapshot:")
    with pd.option_context("display.width", 140, "display.float_format", "{:.3f}".format):
        print(results["latest_year_snapshot"].to_string(index=False))


if __name__ == "__main__":
    main()
```

A3. Python: Excel workbook and dashboard (BI layer) (src/build_workbook.py)

```
"""
Build the Excel workbook: UK_Financial_Health_Review.xlsx

Every ratio in the workbook is a live Excel formula that points back to the
cleaned figures, which in turn point back to the raw extracted figures. Change
a raw number and the ratios and dashboard update.

Run after src/analysis.py:
    python src/build_workbook.py
"""

from __future__ import annotations
```

```
from pathlib import Path

import pandas as pd
from openpyxl import Workbook
from openpyxl.chart import BarChart, LineChart, Reference
from openpyxl.chart.label import DataLabelList
from openpyxl.chart.marker import Marker
from openpyxl.chart.series import DataPoint
from openpyxl.chart.text import RichText
from openpyxl.drawing.text import CharacterProperties, Paragraph, ParagraphProperties, Font as DFont


def chart_title(chart, text: str, size: int = 1100) -> None:
    """Set a smaller, bold chart title in the workbook font."""
    from openpyxl.chart.title import Title
    from openpyxl.chart.text import Text
    from openpyxl.drawing.text import RegularTextRun
    cp = CharacterProperties(sz=size, b=True, solidFill="1F2A44", latin=DFont(typeface="Arial"))
    para = Paragraph(pPr=ParagraphProperties(defRPr=cp), r=[RegularTextRun(rPr=cp, t=text)])
    chart.title = Title(tx=Text(rich=RichText(p=[para])), overlay=False)


def axis_font(axis, size: int = 900) -> None:
    cp = CharacterProperties(sz=size, solidFill="5B6577", latin=DFont(typeface="Arial"))
    axis.txPr = RichText(p=[Paragraph(pPr=ParagraphProperties(defRPr=cp), endParaRPr=cp)])
from openpyxl.comments import Comment
from openpyxl.formatting.rule import CellIsRule
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter
from openpyxl.worksheet.page import PageMargins

ROOT = Path(__file__).resolve().parents[1]
DATA = ROOT / "data"
OUT = ROOT / "outputs"
FILE = OUT / "UK_Financial_Health_Review.xlsx"

# ---------------------------------------------------------------- styling
FONT = "Arial"
INK = "1F2A44"        # dark slate used for headings
MUTED = "5B6577"
LINE = "D5DAE3"
PAPER = "F5F7FA"
WHITE = "FFFFFF"
GOOD_FILL = "E3F2E8"
WARN_FILL = "FDF1DC"
BAD_FILL = "FBE4E4"

F_TITLE = Font(name=FONT, size=18, bold=True, color=WHITE)
F_SUB = Font(name=FONT, size=10, color="DCE3EE")
F_H = Font(name=FONT, size=10, bold=True, color=WHITE)
F_BODY = Font(name=FONT, size=10, color="222222")
F_BOLD = Font(name=FONT, size=10, bold=True, color="222222")
F_INPUT = Font(name=FONT, size=10, color="0000FF")      # hardcoded inputs
F_LINK = Font(name=FONT, size=10, color="008000")       # links to other sheets
F_URL = Font(name=FONT, size=10, color="1155CC", underline="single")
F_SECTION = Font(name=FONT, size=12, bold=True, color=INK)
F_NOTE = Font(name=FONT, size=9, italic=True, color=MUTED)

FILL_INK = PatternFill("solid", fgColor=INK)
FILL_PAPER = PatternFill("solid", fgColor=PAPER)
FILL_WHITE = PatternFill("solid", fgColor=WHITE)
THIN = Side(style="thin", color=LINE)
BOX = Border(left=THIN, right=THIN, top=THIN, bottom=THIN)
WRAP = Alignment(wrap_text=True, vertical="top")
CENTER = Alignment(horizontal="center", vertical="center", wrap_text=True)
RIGHT = Alignment(horizontal="right", vertical="center")

GBP_M = '#,##0.0;(#,##0.0);"-"'
GBP_RAW = '#,##0.0;(#,##0.0);"0"'
PCT = '0.0%;(0.0%);"0.0%"'
TIMES = '0.00"x";(0.00"x");"0.00x"'
COVER = '#,##0.0"x";(#,##0.0"x");"0.0x"'


def style_header(ws, row: int, ncols: int, start_col: int = 1, height: int = 30) -> None:
    for c in range(start_col, start_col + ncols):
        cell = ws.cell(row=row, column=c)
        cell.font = F_H
        cell.fill = FILL_INK
        cell.alignment = CENTER
        cell.border = BOX
    ws.row_dimensions[row].height = height


def title_band(ws, title: str, subtitle: str, ncols: int) -> None:
    for r in (1, 2):
        for c in range(1, ncols + 1):
            ws.cell(row=r, column=c).fill = FILL_INK
    ws["A1"] = title
    ws["A1"].font = F_TITLE
    ws["A1"].alignment = Alignment(vertical="center", indent=1)
    ws["A2"] = subtitle
    ws["A2"].font = F_SUB
    ws["A2"].alignment = Alignment(vertical="top", indent=1)
    ws.row_dimensions[1].height = 34
    ws.row_dimensions[2].height = 22


def set_widths(ws, widths: dict[str, float]) -> None:
    for col, w in widths.items():
```

```
        ws.column_dimensions[col].width = w


def page_setup(ws, landscape: bool = True) -> None:
    ws.page_setup.orientation = "landscape" if landscape else "portrait"
    ws.page_setup.fitToWidth = 1
    ws.page_setup.fitToHeight = 0
    ws.sheet_properties.pageSetUpPr.fitToPage = True
    ws.page_margins = PageMargins(left=0.4, right=0.4, top=0.5, bottom=0.5)
    ws.sheet_view.showGridLines = False


# ---------------------------------------------------------------- data
raw = pd.read_csv(DATA / "raw_financials.csv", dtype={"company_number": str})
raw["fy_end"] = pd.to_datetime(raw["fy_end"])
raw = raw.sort_values(["company", "fy_end"]).reset_index(drop=True)
companies = pd.read_csv(DATA / "companies.csv", dtype={"company_number": str})
sources = pd.read_csv(DATA / "sources.csv")
quality = pd.read_csv(OUT / "data_quality_log.csv")
sql_results = pd.read_excel(OUT / "sql_results.xlsx", sheet_name=None)
BRAND = dict(zip(companies["company"], companies["brand_colour"]))
SHORT = dict(zip(companies["company"], companies["short_name"]))

wb = Workbook()
wb.remove(wb.active)
ws_readme = wb.create_sheet("README")
ws_dash = wb.create_sheet("Dashboard")
ws_kpi = wb.create_sheet("KPI_Scorecard")
ws_story = wb.create_sheet("Project_Story")
ws_co = wb.create_sheet("Companies")
ws_raw = wb.create_sheet("Raw_Data")
ws_cl = wb.create_sheet("Clean_GBPm")
ws_r = wb.create_sheet("Ratios")
ws_g = wb.create_sheet("Formula_Guide")
ws_log = wb.create_sheet("Data_Log")
ws_sql = wb.create_sheet("SQL_Checks")
ws_src = wb.create_sheet("Sources")

FIRST = 5                      # first data row on Raw/Clean/Ratios
LAST = FIRST + len(raw) - 1

# ================================================================ Companies
title_band(ws_co, "Company register",
           "Six UK-listed asset and wealth managers. Company numbers from Companies House. Pages checked 4-5 Nov 2026.",
9)
heads = ["Company", "Short name", "Company number", "Sector", "SIC codes", "Year end",
         "Accounts type", "Latest accounts seen on Companies House", "Companies House link"]
for i, h in enumerate(heads, 1):
    ws_co.cell(row=4, column=i, value=h)
style_header(ws_co, 4, len(heads))
for r, row in enumerate(companies.itertuples(index=False), start=5):
    vals = [row.company, row.short_name, row.company_number, row.sector, row.sic_codes,
            row.year_end, row.accounts_type, row.latest_accounts_seen_on_companies_house,
            row.companies_house_link]
    for c, v in enumerate(vals, 1):
        cell = ws_co.cell(row=r, column=c, value=v)
        cell.font, cell.alignment, cell.border = F_BODY, WRAP, BOX
    ws_co.cell(row=r, column=1).font = Font(name=FONT, size=10, bold=True, color=row.brand_colour)
    ws_co.cell(row=r, column=3).number_format = "@"
    link = ws_co.cell(row=r, column=9)
    link.hyperlink = row.companies_house_link
    link.font = F_URL
    ws_co.cell(row=r, column=1).border = Border(left=Side(style="thick", color=row.brand_colour),
                                                right=THIN, top=THIN, bottom=THIN)
    ws_co.row_dimensions[r].height = 42
note_row = 5 + len(companies) + 1
ws_co.cell(row=note_row, column=1,
           value="Why this sector: asset and wealth managers split their balance sheets into current and "
                 "non-current items, so all five ratios can be calculated. Banks and insurers do not, "
                 "which makes a current ratio meaningless for them.").font = F_NOTE
ws_co.merge_cells(start_row=note_row, start_column=1, end_row=note_row, end_column=9)
ws_co.row_dimensions[note_row].height = 30
ws_co.cell(row=note_row, column=1).alignment = WRAP
set_widths(ws_co, {"A": 30, "B": 14, "C": 15, "D": 30, "E": 14, "F": 13, "G": 22, "H": 34, "I": 58})
ws_co.freeze_panes = "B5"
page_setup(ws_co)

# ================================================================ Raw_Data
raw_heads = ["Company", "Company number", "Financial year", "Year end date", "Units",
             "Revenue", "Net revenue / gross profit", "Operating profit", "Finance income",
             "Finance costs", "Profit before tax", "Profit for the year", "Current assets",
             "Current liabilities", "Borrowings", "Lease liabilities", "Total equity",
             "Source ID", "Notes from the accounts"]
title_band(ws_raw, "Raw data as printed in the accounts",
           "Blue figures are typed in exactly as shown in each source, in the units stated. Blank = not shown; nothing
estimated.",
           len(raw_heads))
for i, h in enumerate(raw_heads, 1):
    ws_raw.cell(row=4, column=i, value=h)
style_header(ws_raw, 4, len(raw_heads), height=42)
money_cols = ["revenue", "net_revenue", "operating_profit", "finance_income", "finance_costs",
              "profit_before_tax", "profit_for_year", "current_assets", "current_liabilities",
              "borrowings", "lease_liabilities", "total_equity"]
for r, row in enumerate(raw.itertuples(index=False), start=FIRST):
    vals = [row.company, row.company_number, row.fy_label, row.fy_end.to_pydatetime(), row.units]
    vals += [None if pd.isna(getattr(row, c)) else float(getattr(row, c)) for c in money_cols]
    vals += [row.source_id, row.notes]
    for c, v in enumerate(vals, 1):
        cell = ws_raw.cell(row=r, column=c, value=v)
        cell.border = BOX
```

```
        cell.font = F_INPUT if 6 <= c <= 17 else F_BODY
        if 6 <= c <= 17:
            cell.number_format = '#,##0.0;(#,##0.0)'
            if v is None:
                cell.fill = PatternFill("solid", fgColor=WARN_FILL)
    ws_raw.cell(row=r, column=1).font = Font(name=FONT, size=10, bold=True, color=BRAND[row.company])
    ws_raw.cell(row=r, column=2).number_format = "@"
    ws_raw.cell(row=r, column=4).number_format = "dd mmm yyyy"
    ws_raw.cell(row=r, column=19).alignment = WRAP
    ws_raw.row_dimensions[r].height = 40
ws_raw.cell(row=LAST + 2, column=1,
            value="Shaded cells: figure not shown in the retrieved statements. Each source ID is listed on the Sources
sheet.").font = F_NOTE
set_widths(ws_raw, {"A": 30, "B": 12, "C": 10, "D": 13, "E": 9, **{get_column_letter(c): 12.5 for c in range(6, 18)},
                    "R": 9, "S": 70})
ws_raw.freeze_panes = "D5"
page_setup(ws_raw)

# ============================================================== Clean_GBPm
clean_heads = ["Company", "Short name", "Financial year", "Year end date", "Source units",
               "Revenue", "Net revenue / gross profit", "Operating profit", "Finance income",
               "Finance costs", "Profit before tax", "Profit for the year", "Current assets",
               "Current liabilities", "Borrowings", "Lease liabilities", "Total equity",
               "Unit factor", "Total debt (borrowings + leases)"]
title_band(ws_cl, "Cleaned data in £ millions",
           "Green = linked to Raw_Data. Figures in £000 are divided by 1,000. Blanks stay blank so no ratio uses a guessed
number.",
           len(clean_heads))
for i, h in enumerate(clean_heads, 1):
    ws_cl.cell(row=4, column=i, value=h)
style_header(ws_cl, 4, len(clean_heads), height=42)
for r in range(FIRST, LAST + 1):
    ws_cl.cell(row=r, column=1, value=f"=Raw_Data!A{r}")
    ws_cl.cell(row=r, column=2, value=f'=IFERROR(INDEX(Companies!$B$5:$B$10,MATCH(A{r},Companies!$A$5:$A$10,0)),A{r})')
    ws_cl.cell(row=r, column=3, value=f"=Raw_Data!C{r}")
    ws_cl.cell(row=r, column=4, value=f"=Raw_Data!D{r}")
    ws_cl.cell(row=r, column=5, value=f"=Raw_Data!E{r}")
    for c in range(6, 18):
        col = get_column_letter(c)
        ws_cl.cell(row=r, column=c, value=f'=IF(Raw_Data!{col}{r}="","",Raw_Data!{col}{r}*$R{r})')
        ws_cl.cell(row=r, column=c).number_format = GBP_RAW
    ws_cl.cell(row=r, column=18, value=f'=IF(E{r}="GBP000",0.001,1)')
    ws_cl.cell(row=r, column=19, value=f'=IF(AND(ISNUMBER(O{r}),ISNUMBER(P{r})),O{r}+P{r},"")')
    ws_cl.cell(row=r, column=19).number_format = GBP_RAW
    for c in range(1, 20):
        cell = ws_cl.cell(row=r, column=c)
        cell.border = BOX
        cell.font = F_LINK if c in (1, 3, 4, 5) or 6 <= c <= 17 else F_BODY
    ws_cl.cell(row=r, column=4).number_format = "dd mmm yyyy"
    ws_cl.cell(row=r, column=18).number_format = "0.000"
set_widths(ws_cl, {"A": 30, "B": 14, "C": 10, "D": 13, "E": 9, **{get_column_letter(c): 12.5 for c in range(6, 18)},
                   "R": 9, "S": 15})
ws_cl.freeze_panes = "D5"
page_setup(ws_cl)

# ================================================================ Ratios
r_heads = ["Company", "Short name", "Financial year", "Year end date", "Revenue £m",
           "Profit before tax £m", "Revenue growth", "Profit margin (PBT)",
           "Operating margin", "Current ratio", "Debt-to-equity", "Interest coverage",
           "Later years for this company", "Use in analysis", "Lookup key"]
title_band(ws_r, "Ratio calculations",
           "All cells are formulas on Clean_GBPm. 'n/a' means the accounts do not give the inputs, or the ratio has no
meaning (e.g. no interest cost).",
           len(r_heads))
for i, h in enumerate(r_heads, 1):
    ws_r.cell(row=4, column=i, value=h)
style_header(ws_r, 4, len(r_heads), height=42)
CL = "Clean_GBPm!"
for r in range(FIRST, LAST + 1):
    f = {
        1: f"={CL}A{r}",
        2: f"={CL}B{r}",
        3: f"={CL}C{r}",
        4: f"={CL}D{r}",
        5: f"={CL}F{r}",
        6: f"={CL}K{r}",
        7: (f'=IF(AND(A{r}=A{r-1},ISNUMBER(E{r}),ISNUMBER(E{r-1})),IF(E{r-1}<>0,E{r}/E{r-1}-1,"n/a"),"n/a")'
            if r > FIRST else '="n/a"'),
        8: f'=IF(AND(ISNUMBER({CL}K{r}),ISNUMBER({CL}F{r})),IF({CL}F{r}<>0,{CL}K{r}/{CL}F{r},"n/a"),"n/a")',
        9: f'=IF(AND(ISNUMBER({CL}H{r}),ISNUMBER({CL}F{r})),IF({CL}F{r}<>0,{CL}H{r}/{CL}F{r},"n/a"),"n/a")',
        10: f'=IF(AND(ISNUMBER({CL}M{r}),ISNUMBER({CL}N{r})),IF({CL}N{r}<>0,{CL}M{r}/{CL}N{r},"n/a"),"n/a")',
        11: f'=IF(AND(ISNUMBER({CL}S{r}),ISNUMBER({CL}Q{r})),IF({CL}Q{r}>0,{CL}S{r}/{CL}Q{r},"n/a"),"n/a")',
        12: f'=IF(AND(ISNUMBER({CL}J{r}),ISNUMBER({CL}K{r})),IF({CL}J{r}>0,({CL}K{r}+{CL}J{r})/{CL}J{r},"n/a"),"n/a")',
        13: f'=COUNTIFS($A${FIRST}:$A${LAST},A{r},$D${FIRST}:$D${LAST},">"&D{r})',
        14: f'=IF(M{r}<=2,"Analysed","Base year (growth only)")',
        15: f'=A{r}&"|"&M{r}',
    }
    for c, formula in f.items():
        cell = ws_r.cell(row=r, column=c, value=formula)
        cell.border = BOX
        cell.font = F_LINK if c <= 6 else F_BODY
        cell.alignment = RIGHT if 5 <= c <= 12 else Alignment(vertical="center")
    ws_r.cell(row=r, column=4).number_format = "dd mmm yyyy"
    ws_r.cell(row=r, column=5).number_format = GBP_M
    ws_r.cell(row=r, column=6).number_format = GBP_M
    for c in (7, 8, 9):
        ws_r.cell(row=r, column=c).number_format = PCT
    ws_r.cell(row=r, column=10).number_format = TIMES
    ws_r.cell(row=r, column=11).number_format = TIMES
    ws_r.cell(row=r, column=12).number_format = COVER
```

```
    ws_r.cell(row=r, column=15).font = Font(name=FONT, size=8, color="9AA3B2")
# light banding by company, brand colour on company name
for r, row in enumerate(raw.itertuples(index=False), start=FIRST):
    ws_r.cell(row=r, column=2).font = Font(name=FONT, size=10, bold=True, color=BRAND[row.company])
    if list(companies["company"]).index(row.company) % 2 == 0:
        for c in range(1, 16):
            ws_r.cell(row=r, column=c).fill = FILL_PAPER
# conditional colours (only numeric cells are affected)
rng = lambda col: f"{col}{FIRST}:{col}{LAST}"
ws_r.conditional_formatting.add(rng("H"), CellIsRule(operator="lessThan", formula=["0.05"], fill=PatternFill("solid",
fgColor=BAD_FILL)))
ws_r.conditional_formatting.add(rng("H"), CellIsRule(operator="greaterThanOrEqual", formula=["0.25"],
fill=PatternFill("solid", fgColor=GOOD_FILL)))
ws_r.conditional_formatting.add(rng("G"), CellIsRule(operator="lessThan", formula=["0"], font=Font(name=FONT,
color="B42318")))
ws_r.conditional_formatting.add(rng("J"), CellIsRule(operator="lessThan", formula=["1.5"], fill=PatternFill("solid",
fgColor=WARN_FILL)))
ws_r.cell(row=LAST + 2, column=1,
          value="Shading: green margin = 25% or more; red margin = under 5%; amber current ratio = under 1.5x; red growth
= revenue fell. "
                "These bands are reading aids, not pass/fail tests.").font = F_NOTE
ws_r.cell(row=LAST + 3, column=1,
          value="Interest coverage uses profit before tax plus finance costs, divided by finance costs. A negative result
means earnings did not cover interest that year.").font = F_NOTE
ws_r.cell(row=7, column=12).comment = None
set_widths(ws_r, {"A": 30, "B": 14, "C": 10, "D": 13, "E": 12, "F": 13, "G": 12, "H": 13,
                  "I": 12, "J": 11, "K": 12, "L": 12, "M": 11, "N": 20, "O": 6})
ws_r.column_dimensions["O"].hidden = False
ws_r.freeze_panes = "D5"
page_setup(ws_r)

# ============================================================== Dashboard
order = (sql_results["latest_year_snapshot"]["company"].tolist())  # sorted by PBT margin
ws = ws_dash
ncol = 12
title_band(ws, "UK Asset & Wealth Managers: Financial Health Review",
           "Latest audited year for each company, figures in £ millions. Built from Companies House-listed group accounts;
data accessed 4-5 Nov 2026.",
           ncol)
ws.row_dimensions[3].height = 8

# --- KPI table
ws["A4"] = "Latest-year scorecard (sorted by profit margin)"
ws["A4"].font = F_SECTION
heads = ["Company", "Latest year", "Revenue £m", "Revenue growth", "Profit margin (PBT)",
         "Current ratio", "Debt-to-equity", "Interest coverage", "What stands out"]
for i, h in enumerate(heads, 1):
    ws.cell(row=5, column=i, value=h)
style_header(ws, 5, len(heads), height=34)
ws.merge_cells(start_row=5, start_column=9, end_row=5, end_column=12)

READ = {
    "AJ Bell plc": "Steady growth, high margin, no borrowings. Check how much profit relies on interest earned on client
cash.",
    "Ashmore Group plc": "Most liquid balance sheet and no debt, but revenue has fallen three years running. Profit leans
on investment income.",
    "Polar Capital Holdings plc": "Revenue up three years in a row, no borrowings. Lease debt not shown separately, so
debt-to-equity is incomplete.",
    "Jupiter Fund Management plc": "Recovered from a 2023 goodwill write-down. Repaid its £50m notes in April 2025.",
    "Liontrust Asset Management plc": "Revenue down three years running and equity has halved since FY2023. Lowest
liquidity cushion of the profitable group.",
    "Premier Miton Group plc": "No debt, but profit margin is thin and still falling. Revenue has dropped about 30% since
FY2022.",
}
R = "Ratios!"
KEYRANGE = f"{R}$O${FIRST}:$O${LAST}"


def lookup(col: str, key_cell: str) -> str:
    return f'=IFERROR(INDEX({R}${col}${FIRST}:${col}${LAST},MATCH({key_cell},{KEYRANGE},0)),"n/a")'


for i, company in enumerate(order):
    r = 6 + i
    ws.cell(row=r, column=1, value=company)
    ws.cell(row=r, column=13, value=f'=A{r}&"|0"')          # helper key (hidden col M)
    ws.cell(row=r, column=2, value=lookup("C", f"$M{r}"))
    ws.cell(row=r, column=3, value=lookup("E", f"$M{r}"))
    ws.cell(row=r, column=4, value=lookup("G", f"$M{r}"))
    ws.cell(row=r, column=5, value=lookup("H", f"$M{r}"))
    ws.cell(row=r, column=6, value=lookup("J", f"$M{r}"))
    ws.cell(row=r, column=7, value=lookup("K", f"$M{r}"))
    ws.cell(row=r, column=8, value=lookup("L", f"$M{r}"))
    ws.cell(row=r, column=9, value=READ[company])
    ws.merge_cells(start_row=r, start_column=9, end_row=r, end_column=12)
    brand = BRAND[company]
    for c in range(1, 13):
        cell = ws.cell(row=r, column=c)
        cell.border = BOX
        cell.font = F_BODY
        cell.alignment = Alignment(vertical="center", horizontal="right" if 3 <= c <= 8 else "left",
                                   wrap_text=(c == 9), indent=1 if c in (1, 9) else 0)
    ws.cell(row=r, column=1).font = Font(name=FONT, size=11, bold=True, color=WHITE)
    ws.cell(row=r, column=1).fill = PatternFill("solid", fgColor=brand)
    ws.cell(row=r, column=2).alignment = CENTER
    ws.cell(row=r, column=3).number_format = GBP_M
    ws.cell(row=r, column=4).number_format = PCT
    ws.cell(row=r, column=5).number_format = PCT
    ws.cell(row=r, column=6).number_format = TIMES
    ws.cell(row=r, column=7).number_format = TIMES
```

```
    ws.cell(row=r, column=8).number_format = COVER
    ws.row_dimensions[r].height = 36
last_kpi = 6 + len(order) - 1
ws.column_dimensions["M"].hidden = True
kr = lambda col: f"{col}6:{col}{last_kpi}"
ws.conditional_formatting.add(kr("D"), CellIsRule(operator="lessThan", formula=["0"], font=Font(name=FONT, bold=True,
color="B42318"), fill=PatternFill("solid", fgColor=BAD_FILL)))
ws.conditional_formatting.add(kr("D"), CellIsRule(operator="greaterThan", formula=["0.1"], font=Font(name=FONT, bold=True,
color="1E7B3A"), fill=PatternFill("solid", fgColor=GOOD_FILL)))
ws.conditional_formatting.add(kr("E"), CellIsRule(operator="greaterThanOrEqual", formula=["0.25"], font=Font(name=FONT,
bold=True, color="1E7B3A"), fill=PatternFill("solid", fgColor=GOOD_FILL)))
ws.conditional_formatting.add(kr("E"), CellIsRule(operator="lessThan", formula=["0.05"], font=Font(name=FONT, bold=True,
color="B42318"), fill=PatternFill("solid", fgColor=BAD_FILL)))
ws.conditional_formatting.add(kr("F"), CellIsRule(operator="lessThan", formula=["1.5"], fill=PatternFill("solid",
fgColor=WARN_FILL)))
note_r = last_kpi + 1
ws.cell(row=note_r, column=1,
        value="Latest years differ: Liontrust and Polar Capital to Mar 2026; AJ Bell and Premier Miton to Sep 2025;
Ashmore to Jun 2025; Jupiter to Dec 2025. "
              "Interest coverage is n/a where a company reports no interest cost.").font = F_NOTE
ws.merge_cells(start_row=note_r, start_column=1, end_row=note_r, end_column=12)
ws.row_dimensions[note_r].height = 26
ws.cell(row=note_r, column=1).alignment = WRAP

# --- trend helper tables (feed the charts)
helper_top = note_r + 2
ws.cell(row=helper_top, column=1, value="Three-year trend (chart data)").font = F_SECTION
trend_specs = [("Profit margin (PBT)", "H", PCT), ("Revenue growth", "G", PCT), ("Current ratio", "J", TIMES)]
blocks = {}
row_ptr = helper_top + 1
for label, col, fmt in trend_specs:
    hdr = row_ptr
    for i, h in enumerate([label, "Two years before", "One year before", "Latest year"], 1):
        ws.cell(row=hdr, column=i, value=h)
    style_header(ws, hdr, 4, height=22)
    for i, company in enumerate(order):
        r = hdr + 1 + i
        ws.cell(row=r, column=1, value=SHORT[company])
        ws.cell(row=r, column=1).font = Font(name=FONT, size=10, bold=True, color=BRAND[company])
        for j, rank in enumerate((2, 1, 0)):
            c = ws.cell(row=r, column=2 + j,
                        value=f'=IFERROR(INDEX({R}${col}${FIRST}:${col}${LAST},MATCH("{company}|{rank}",
{KEYRANGE},0)),"n/a")')
            c.number_format = fmt
            c.font = F_BODY
            c.alignment = RIGHT
        for cc in range(1, 5):
            ws.cell(row=r, column=cc).border = BOX
    blocks[label] = (hdr, hdr + len(order))
    row_ptr = hdr + len(order) + 2
# years behind each column, for reference
yr_hdr = row_ptr
for i, h in enumerate(["Financial years used", "Two years before", "One year before", "Latest year"], 1):
    ws.cell(row=yr_hdr, column=i, value=h)
style_header(ws, yr_hdr, 4, height=22)
for i, company in enumerate(order):
    r = yr_hdr + 1 + i
    ws.cell(row=r, column=1, value=SHORT[company]).font = Font(name=FONT, size=10, bold=True, color=BRAND[company])
    for j, rank in enumerate((2, 1, 0)):
        c = ws.cell(row=r, column=2 + j,
                    value=f'=IFERROR(INDEX({R}$C${FIRST}:$C${LAST},MATCH("{company}|{rank}",{KEYRANGE},0)),"n/a")')
        c.font, c.alignment = F_BODY, CENTER
    for cc in range(1, 5):
        ws.cell(row=r, column=cc).border = BOX

# --- charts
def line_chart(label: str, title: str, y_fmt: str) -> LineChart:
    hdr, end = blocks[label]
    ch = LineChart()
    chart_title(ch, title)
    ch.height, ch.width = 7.6, 17.5
    ch.y_axis.number_format = y_fmt
    ch.y_axis.majorGridlines = None
    ch.y_axis.delete = False
    ch.x_axis.delete = False
    data = Reference(ws, min_col=1, max_col=4, min_row=hdr + 1, max_row=end)
    ch.add_data(data, from_rows=True, titles_from_data=True)
    ch.set_categories(Reference(ws, min_col=2, max_col=4, min_row=hdr, max_row=hdr))
    for s, company in zip(ch.series, order):
        colour = BRAND[company]
        s.graphicalProperties.line.solidFill = colour
        s.graphicalProperties.line.width = 28000
        s.marker = Marker(symbol="circle", size=7)
        s.marker.graphicalProperties.solidFill = colour
        s.marker.graphicalProperties.line.solidFill = colour
        s.smooth = False
    ch.x_axis.tickLblPos = "low"
    axis_font(ch.x_axis)
    axis_font(ch.y_axis)
    ch.legend.position = "b"
    return ch


ws.add_chart(line_chart("Profit margin (PBT)", "Profit before tax margin, last three years", "0%"), "F15")
ws.add_chart(line_chart("Revenue growth", "Revenue growth, last three years", "0%"), "F31")

hdr, end = blocks["Current ratio"]
bar = BarChart()
bar.type = "bar"
chart_title(bar, "Current ratio, latest year (current assets ÷ current liabilities)")
```

```
bar.height, bar.width = 7.6, 17.5
axis_font(bar.x_axis)
axis_font(bar.y_axis)
bar.add_data(Reference(ws, min_col=4, min_row=hdr + 1, max_row=end), titles_from_data=False)
bar.set_categories(Reference(ws, min_col=1, min_row=hdr + 1, max_row=end))
bar.legend = None
bar.y_axis.number_format = '0.0"x"'
bar.y_axis.majorGridlines = None
bar.y_axis.delete = False
bar.x_axis.delete = False
bar.x_axis.scaling.orientation = "maxMin"
s = bar.series[0]
for idx, company in enumerate(order):
    pt = DataPoint(idx=idx)
    pt.graphicalProperties.solidFill = BRAND[company]
    pt.graphicalProperties.line.solidFill = BRAND[company]
    s.dPt.append(pt)
s.dLbls = DataLabelList()
s.dLbls.showVal = True
s.dLbls.showSerName = False
s.dLbls.showCatName = False
s.dLbls.showLegendKey = False
s.dLbls.showPercent = False
s.dLbls.numFmt = '0.0"x"'
bar.gapWidth = 60
ws.add_chart(bar, "F47")

# --- key findings box beneath helper tables
kf_top = yr_hdr + len(order) + 2
ws.cell(row=kf_top, column=1, value="Key points for a lender or investor").font = F_SECTION
points = [
    "AJ Bell combines the strongest growth (revenue +18% to +33% a year) with a rising margin (40% to 43%) and no
borrowings.",
    "Ashmore and Polar Capital also carry no borrowings. Ashmore's very high margin depends on investment income, while
its fee revenue fell 24% in FY2025.",
    "Jupiter's margin recovered from 2.3% (FY2023, after a £76m goodwill write-down) to 28.3%, and it repaid its only
borrowings in 2025.",
    "Liontrust and Premier Miton show the weakest trends: falling revenue, thin margins and current ratios near 1.3x.",
    "None of these ratios proves a firm is safe or at risk. They show where to ask questions next.",
]
for i, p in enumerate(points):
    r = kf_top + 1 + i
    ws.cell(row=r, column=1, value=f"•  {p}")
    ws.merge_cells(start_row=r, start_column=1, end_row=r, end_column=4)
    ws.cell(row=r, column=1).font = F_BODY
    ws.cell(row=r, column=1).alignment = WRAP
    ws.row_dimensions[r].height = 44

set_widths(ws, {"A": 36, "B": 14, "C": 13, "D": 13, "E": 13, "F": 12, "G": 13, "H": 13,
                "I": 18, "J": 18, "K": 18, "L": 18})
ws.freeze_panes = "A4"
page_setup(ws)
ws.sheet_view.zoomScale = 90

# ============================================================== Formula guide
title_band(ws_g, "How each ratio is worked out",
           "Plain-English formulas, what a higher or lower result may suggest, and why no ratio should be read alone.", 6)
g_heads = ["Ratio", "Formula in plain words", "Excel formula used (row 5 example)",
           "A higher result may indicate", "A lower result may indicate", "Why not to read it on its own"]
for i, h in enumerate(g_heads, 1):
    ws_g.cell(row=4, column=i, value=h)
style_header(ws_g, 4, 6)
guide = [
    ("Revenue growth", "This year's revenue divided by last year's revenue, minus 1.",
     "=E6/E5-1 (same company, consecutive years)",
     "The firm is winning business or benefiting from rising markets.",
     "Clients are leaving, fees are being cut or markets have fallen.",
     "For fund managers, revenue moves with stock and bond markets, so growth can reflect market levels rather than
management skill."),
    ("Profit margin (profit before tax)", "Profit before tax divided by revenue. We use profit before tax because Polar
Capital does not show an operating profit line, so it is the one measure every company reports.",
     "=Clean_GBPm!K5/Clean_GBPm!F5",
     "More of each £1 of revenue is kept as profit; costs are under control.",
     "Costs are high relative to income, or one-off charges hit the year.",
     "It includes investment income and one-off items. Ashmore's margin is lifted by returns on its own investments;
Jupiter's FY2023 margin is cut by a goodwill write-down. Operating margin is shown alongside as a check."),
    ("Current ratio", "Current assets divided by current liabilities.",
     "=Clean_GBPm!M5/Clean_GBPm!N5",
     "More short-term assets than bills due within a year; a bigger cushion.",
     "Less room to meet short-term obligations.",
     "Fund managers often hold money due to and from their funds in receivables and payables. These balances can be large
and roughly cancel out, pulling the ratio towards 1x without signalling stress. Too high can also mean idle cash."),
    ("Debt-to-equity", "Borrowings plus lease liabilities, divided by total equity.",
     "=Clean_GBPm!S5/Clean_GBPm!Q5",
     "More reliance on borrowed money, so more fixed obligations.",
     "The firm is funded mostly by shareholders' money.",
     "Most of these firms have no bank debt, so the figure is mainly office leases. It does not capture regulatory capital
needs, pension deficits or seed capital commitments."),
    ("Interest coverage", "Profit before tax plus finance costs, divided by finance costs (how many times earnings cover
interest).",
     "=(Clean_GBPm!K5+Clean_GBPm!J5)/Clean_GBPm!J5",
     "Interest is easily covered by earnings.",
     "Earnings would struggle to pay interest; a negative figure means a loss.",
     "When interest costs are tiny (lease interest of a few hundred thousand pounds), the ratio becomes very large and
adds little. It is n/a where no interest cost is reported."),
]
for r, row in enumerate(guide, start=5):
    for c, v in enumerate(row, 1):
        cell = ws_g.cell(row=r, column=c, value=v)
        if c == 3:
```

```
            cell.data_type = "s"   # show the formula as text, not run it
            cell.font = Font(name="Consolas", size=10, color=INK)
        else:
            cell.font = F_BOLD if c == 1 else F_BODY
        cell.alignment = WRAP
        cell.border = BOX
    ws_g.row_dimensions[r].height = 92
ws_g.cell(row=11, column=1,
          value="Operating margin (supplementary): operating profit divided by revenue. Shown on the Ratios sheet to
separate the core fee business from investment returns and one-offs.").font = F_NOTE
ws_g.merge_cells("A11:F11")
set_widths(ws_g, {"A": 22, "B": 38, "C": 30, "D": 30, "E": 30, "F": 52})
ws_g.freeze_panes = "B5"
page_setup(ws_g)

# ============================================================== Data log
title_band(ws_log, "Data checks: missing values, unusual figures and comparability",
           "Generated by src/analysis.py, plus analyst notes. Nothing has been estimated or filled in.", 5)
l_heads = ["Company", "Financial year", "Type", "Field", "Detail"]
for i, h in enumerate(l_heads, 1):
    ws_log.cell(row=4, column=i, value=h)
style_header(ws_log, 4, 5)
extra = pd.DataFrame([
    ("All companies", "All", "Comparability", "year end",
     "Year ends differ (March, June, September, December). Latest years range from Jun 2025 to Mar 2026, so market
conditions are not identical."),
    ("All companies", "All", "Source", "source document",
     "Companies House accounts PDFs could not be opened in this environment. Figures were taken from the same audited
group statements published by each company (annual report or audited results announcement). Each row cites its source."),
    ("Ashmore Group plc", "FY2026", "Missing year", "all fields",
     "FY2026 (year to 30 Jun 2026) results exist but the full statements could not be retrieved, so FY2023-FY2025 are
analysed."),
    ("AJ Bell plc", "All", "Source", "source document",
     "Taken from audited results announcements, not the full annual report, which could not be read past the auditor's
report."),
    ("Polar Capital Holdings plc", "All", "Missing value", "lease_liabilities",
     "Lease liabilities are within other balance sheet lines, so debt-to-equity is shown as n/a rather than an understated
0."),
    ("Jupiter Fund Management plc", "FY2023", "Unusual figure", "operating_profit",
     "Operating profit (86.0) is before a 76.2 goodwill impairment, which is shown as a separate line."),
    ("Liontrust Asset Management plc", "All", "Comparability", "net_revenue",
     "Net revenue column holds gross profit (revenue less cost of sales), the nearest equivalent line."),
], columns=l_heads[:4] + ["Detail"])
extra.columns = ["company", "fy_label", "type", "field", "detail"]
log_all = pd.concat([extra, quality], ignore_index=True)
for r, row in enumerate(log_all.itertuples(index=False), start=5):
    for c, v in enumerate(row, 1):
        cell = ws_log.cell(row=r, column=c, value=v)
        cell.font, cell.alignment, cell.border = F_BODY, WRAP, BOX
    fill = {"Missing value": WARN_FILL, "Missing year": WARN_FILL, "Unusual figure": BAD_FILL}.get(row.type)
    if fill:
        ws_log.cell(row=r, column=3).fill = PatternFill("solid", fgColor=fill)
    ws_log.row_dimensions[r].height = 30 if len(str(row.detail)) < 110 else 44
set_widths(ws_log, {"A": 30, "B": 12, "C": 16, "D": 20, "E": 100})
ws_log.freeze_panes = "A5"
ws_log.auto_filter.ref = f"A4:E{4 + len(log_all)}"
page_setup(ws_log)

# ============================================================== SQL checks
title_band(ws_sql, "SQL checks (results)",
           "Output of sql/queries.sql run on outputs/financials.db by src/analysis.py. Static values, refreshed when the
script runs.", 9)
r = 4
labels = {
    "latest_year_snapshot": "Latest year for each company",
    "three_year_average": "Three-year averages",
    "revenue_declines": "Years where revenue fell",
    "loss_or_thin_margin_years": "Loss or margin below 5%",
    "missing_inputs": "Ratios that could not be calculated",
}
pct_cols = {"revenue_growth", "pbt_margin", "avg_revenue_growth", "avg_pbt_margin"}
x_cols = {"current_ratio", "debt_to_equity", "interest_cover", "avg_current_ratio", "lowest_current_ratio"}
for name, frame in sql_results.items():
    ws_sql.cell(row=r, column=1, value=labels.get(name, name)).font = F_SECTION
    r += 1
    for i, h in enumerate(frame.columns, 1):
        ws_sql.cell(row=r, column=i, value=h.replace("_", " "))
    style_header(ws_sql, r, len(frame.columns), height=28)
    for row in frame.itertuples(index=False):
        r += 1
        for i, v in enumerate(row, 1):
            col = frame.columns[i - 1]
            val = None if pd.isna(v) else v
            cell = ws_sql.cell(row=r, column=i, value=val if val is not None else "n/a")
            cell.font, cell.border = F_BODY, BOX
            if col in pct_cols:
                cell.number_format = PCT
            elif col in x_cols:
                cell.number_format = COVER if col == "interest_cover" else TIMES
            elif col.endswith("_m"):
                cell.number_format = GBP_M
    r += 2
set_widths(ws_sql, {"A": 30, "B": 16, "C": 16, "D": 16, "E": 16, "F": 16, "G": 16, "H": 16})
ws_sql.freeze_panes = "A4"
page_setup(ws_sql)

# ============================================================== Sources
title_band(ws_src, "Sources",
           "Every figure traces to one of these documents. All accessed 4-5 November 2026.", 6)
s_heads = ["Source ID", "Company", "Document", "Years covered", "Link", "Accessed"]
```

```
for i, h in enumerate(s_heads, 1):
    ws_src.cell(row=4, column=i, value=h)
style_header(ws_src, 4, 6)
for r, row in enumerate(sources.itertuples(index=False), start=5):
    vals = [row.source_id, row.company, row.document, row.years_covered, row.url, row.accessed]
    for c, v in enumerate(vals, 1):
        cell = ws_src.cell(row=r, column=c, value=v)
        cell.font, cell.alignment, cell.border = F_BODY, WRAP, BOX
    ws_src.cell(row=r, column=5).hyperlink = row.url
    ws_src.cell(row=r, column=5).font = F_URL
    ws_src.row_dimensions[r].height = 44
r0 = 5 + len(sources) + 1
ws_src.cell(row=r0, column=1, value="Companies House pages (company numbers, filing records)").font = F_SECTION
for i, row in enumerate(companies.itertuples(index=False), start=r0 + 1):
    ws_src.cell(row=i, column=2, value=row.company).font = F_BODY
    c = ws_src.cell(row=i, column=5, value=row.companies_house_link)
    c.hyperlink, c.font = row.companies_house_link, F_URL
    ws_src.cell(row=i, column=3, value=row.check_note).font = F_BODY
    ws_src.cell(row=i, column=3).alignment = WRAP
    ws_src.row_dimensions[i].height = 30
set_widths(ws_src, {"A": 10, "B": 30, "C": 48, "D": 16, "E": 80, "F": 12})
ws_src.freeze_panes = "A5"
page_setup(ws_src)

# ============================================================ Project story
title_band(ws_story, "Project story: from raw data to stakeholder brief",
           "How the review was built, step by step, and what it means for the people who will use it.", 3)
STORY = [
    ("1. Objective", [
        ("Main question", "Which UK asset and wealth managers look financially stronger, and what risks should a lender or
investor examine?"),
        ("Why this sector", "These firms report current and non-current items, so all five ratios can be calculated. Banks
and insurers do not."),
        ("What it is not", "Not investment advice. It does not claim any firm is safe or likely to fail; it shows where to
ask questions."),
    ]),
    ("2. Raw data", [
        ("Companies", "AJ Bell (04503206), Ashmore (03675683), Jupiter (06150195), Liontrust (02954692), Polar Capital
(04235369), Premier Miton (06306664)."),
        ("Source route", "Identified on Companies House. Figures taken from each company's published audited group
statements, because Companies House PDFs could not be opened here. Accessed 4-5 Nov 2026."),
        ("What was collected", "24 company-years (6 x 4), 12 figures each, typed in exactly as printed. See Raw_Data (blue
cells) and Sources."),
    ]),
    ("3. Data cleaning", [
        ("Common unit", "£000 figures divided by 1,000, so all figures are in £ millions (Clean_GBPm)."),
        ("Matching line names", "'Gross profit' and 'Net income' grouped as net revenue. Debt = borrowings + lease
liabilities."),
        ("No guessing", "Missing figures stay blank; any ratio needing them shows n/a."),
    ]),
    ("4. Exploration", [
        ("Scale", "Latest revenue ranged from £63.3m (Premier Miton) to £465.7m (Jupiter); about £1.39bn in total."),
        ("Direction", "11 of 18 analysed company-years had falling revenue. Median growth was -1.6%. Only AJ Bell and
Polar Capital grew every year."),
        ("Spread", "PBT margins ran from -0.3% to 75.2% (median 25.6%). Current ratios were all above 1.2x (median
2.5x)."),
    ]),
    ("5. Data quality", [
        ("Checks", "18 missing values and 8 unusual figures logged (Data_Log). Nothing estimated."),
        ("Accuracy", "All 144 ratio cells match the Python calculation. 0 formula errors."),
        ("Main gaps", "Polar Capital: no operating profit or lease line. Ashmore: no finance cost line and FY2026 not
retrieved. AJ Bell: results announcements used."),
    ]),
    ("6. Analysis", [
        ("Stronger", "AJ Bell: growth of 33%, 24%, 18% and a 43.4% margin. Polar Capital: growth every year, 29.1% margin.
Jupiter: margin up from 2.3% to 28.3%, £50m notes repaid."),
        ("Mixed", "Ashmore: 7.0x current ratio and no debt, but revenue down to £144.4m, and 47% of profit from investment
income."),
        ("Weaker", "Liontrust: revenue down 45% since FY2023 and equity halved. Premier Miton: margin down to 3.7%."),
        ("KPI scorecard", "Seven KPIs rated green / amber / red (KPI_Scorecard sheet). Screening scores: AJ Bell 100%,
Jupiter 100%, Polar Capital 100% (5 of 7 KPIs), Ashmore 67%, Liontrust 43%, Premier Miton 17%."),
    ]),
    ("7. Recommendations", [
        ("Lender", "Look past debt ratios to covenants, guarantees and pensions. Ask what makes up current assets and
liabilities. Test the regulatory capital surplus against a further 20% revenue fall."),
        ("Investor", "Separate fee profit from investment gains. Track assets under management and net flows each quarter.
Check dividend cover where equity is falling."),
        ("Next update", "Add Ashmore FY2026 and Polar Capital lease figures when available."),
    ]),
    ("8. Business strategy", [
        ("AJ Bell", "How would margins hold if UK interest rates fall and interest on client cash drops?"),
        ("Polar Capital", "How dependent is revenue on a few technology funds and on performance fees?"),
        ("Jupiter", "Can cost discipline hold now that revenue is growing again?"),
        ("Ashmore", "How will fee income be rebuilt, and what is the plan for its large capital base?"),
        ("Liontrust", "Is the cost base shrinking as fast as revenue, and is the dividend covered?"),
        ("Premier Miton", "Is the business large enough to stand alone at a sustainable margin?"),
    ]),
    ("9. Stakeholder brief", [
        ("Situation", "Six UK-listed asset and wealth managers compared over their last three audited years."),
        ("Finding", "None relies on bank debt and all can meet short-term bills. The difference is direction: AJ Bell,
Polar Capital and Jupiter are growing; Ashmore, Liontrust and Premier Miton are shrinking."),
        ("Why it matters", "The risk is not repaying a loan today. It is revenue falling until margins, dividends and
capital buffers come under strain."),
        ("Suggested action", "Treat AJ Bell, Polar Capital and Jupiter as the stronger names on current evidence. Ask the
others for recent fund flows, cost plans and capital surplus before committing."),
    ]),
]
row = 4
for section, items in STORY:
    c = ws_story.cell(row=row, column=1, value=section)
```

```
    c.font = Font(name=FONT, size=11, bold=True, color=WHITE)
    for col in (1, 2, 3):
        ws_story.cell(row=row, column=col).fill = FILL_INK
    ws_story.row_dimensions[row].height = 22
    row += 1
    for label, text in items:
        a = ws_story.cell(row=row, column=1, value=label)
        b = ws_story.cell(row=row, column=2, value=text)
        ws_story.merge_cells(start_row=row, start_column=2, end_row=row, end_column=3)
        brand_hit = next((co for co in companies["company"] if SHORT[co] == label), None)
        a.font = Font(name=FONT, size=10, bold=True, color=BRAND[brand_hit] if brand_hit else INK)
        b.font = F_BODY
        a.alignment = b.alignment = WRAP
        a.border = b.border = BOX
        ws_story.row_dimensions[row].height = 30 if len(text) < 120 else 44
        row += 1
    row += 1
ws_story.cell(row=row, column=1, value="Full detail: README.md and the report in the report folder.").font = F_NOTE
set_widths(ws_story, {"A": 24, "B": 90, "C": 30})
ws_story.freeze_panes = "A4"
page_setup(ws_story, landscape=False)

# ============================================================== KPI scorecard
bands = pd.read_csv(DATA / "kpi_thresholds.csv")
kpi_order = pd.read_csv(OUT / "kpi_scorecard.csv")["company"].tolist()
ws = ws_kpi
n_k = len(bands)
total_cols = 2 + 2 * n_k + 2
title_band(ws, "KPI scorecard: latest year, green / amber / red",
           "Each KPI is a formula on the Ratios sheet. Change a yellow band below and every status updates. Bands are
reading aids, not industry rules.",
           total_cols)

# --- bands (editable inputs)
ws["A4"] = "KPI bands (edit the yellow cells)"
ws["A4"].font = F_SECTION
b_heads = ["KPI", "Better when", "Green from", "Amber from", "Why this band"]
for i, h in enumerate(b_heads, 1):
    ws.cell(row=5, column=i, value=h)
style_header(ws, 5, 4, height=24)
ws.cell(row=5, column=5).font, ws.cell(row=5, column=5).fill = F_H, FILL_INK
ws.merge_cells(start_row=5, start_column=5, end_row=5, end_column=12)
for c in range(5, 13):
    ws.cell(row=5, column=c).fill = FILL_INK
YELLOW = PatternFill("solid", fgColor="FFF6CC")
pct_kpis = {"revenue_growth", "pbt_margin", "operating_margin", "equity_change_3y"}
band_row = {}
for i, b in enumerate(bands.itertuples(), start=6):
    band_row[b.kpi] = i
    fmt = PCT if b.kpi in pct_kpis else '0.00"x"'
    vals = [b.label, b.better, b.green_from, b.amber_from, b.why_this_band]
    for c, v in enumerate(vals, 1):
        cell = ws.cell(row=i, column=c, value=v)
        cell.border = BOX
        cell.font = F_BODY
        cell.alignment = Alignment(vertical="center", wrap_text=(c == 5))
    for c in (3, 4):
        ws.cell(row=i, column=c).fill = YELLOW
        ws.cell(row=i, column=c).font = F_INPUT
        ws.cell(row=i, column=c).number_format = fmt
        ws.cell(row=i, column=c).alignment = CENTER
    ws.cell(row=i, column=2).alignment = CENTER
    ws.merge_cells(start_row=i, start_column=5, end_row=i, end_column=12)
    for c in range(5, 13):
        ws.cell(row=i, column=c).border = BOX
    ws.row_dimensions[i].height = 22
ws.cell(row=6 + n_k, column=1,
        value="'Higher' KPIs: green at or above 'Green from', amber at or above 'Amber from', otherwise red. "
              "'Lower' KPIs (debt-to-equity): green at or below 'Green from', amber at or below 'Amber from'.").font =
F_NOTE

# --- scorecard
top = 6 + n_k + 3
ws.cell(row=top - 1, column=1, value="Latest-year KPI scorecard (sorted by screening score)").font = F_SECTION
ws.cell(row=top, column=1, value="Company")
ws.cell(row=top, column=2, value="Latest year")
ws.merge_cells(start_row=top, start_column=1, end_row=top + 1, end_column=1)
ws.merge_cells(start_row=top, start_column=2, end_row=top + 1, end_column=2)
ratio_col = {"revenue_growth": "G", "pbt_margin": "H", "operating_margin": "I",
             "current_ratio": "J", "debt_to_equity": "K", "interest_cover": "L"}
for k, b in enumerate(bands.itertuples()):
    c = 3 + 2 * k
    ws.cell(row=top, column=c, value=b.label)
    ws.merge_cells(start_row=top, start_column=c, end_row=top, end_column=c + 1)
    ws.cell(row=top + 1, column=c, value="Value")
    ws.cell(row=top + 1, column=c + 1, value="Status")
sc = 3 + 2 * n_k
ws.cell(row=top, column=sc, value="KPIs with data")
ws.cell(row=top, column=sc + 1, value="Screening score")
ws.merge_cells(start_row=top, start_column=sc, end_row=top + 1, end_column=sc)
ws.merge_cells(start_row=top, start_column=sc + 1, end_row=top + 1, end_column=sc + 1)
style_header(ws, top, total_cols, height=34)
style_header(ws, top + 1, total_cols, height=18)
KEY = f"{R}$O${FIRST}:$O${LAST}"
status_cells_all = []
for i, company in enumerate(kpi_order):
    r = top + 2 + i
    ws.cell(row=r, column=1, value=company)
    ws.cell(row=r, column=1).font = Font(name=FONT, size=10, bold=True, color=WHITE)
    ws.cell(row=r, column=1).fill = PatternFill("solid", fgColor=BRAND[company])
```

```
    ws.cell(row=r, column=2, value=f'=IFERROR(INDEX({R}$C${FIRST}:$C${LAST},MATCH($A{r}&"|0",{KEY},0)),"n/a")')
    status_cells = []
    for k, b in enumerate(bands.itertuples()):
        vc, stc = 3 + 2 * k, 4 + 2 * k
        vref = f"{get_column_letter(vc)}{r}"
        if b.kpi == "equity_change_3y":
            f = (f'=IFERROR(INDEX(Clean_GBPm!$Q${FIRST}:$Q${LAST},MATCH($A{r}&"|0",{KEY},0))'
                 f'/INDEX(Clean_GBPm!$Q${FIRST}:$Q${LAST},MATCH($A{r}&"|3",{KEY},0))-1,"n/a")')
        else:
            col = ratio_col[b.kpi]
            f = f'=IFERROR(INDEX({R}${col}${FIRST}:${col}${LAST},MATCH($A{r}&"|0",{KEY},0)),"n/a")'
        ws.cell(row=r, column=vc, value=f)
        br = band_row[b.kpi]
        ws.cell(row=r, column=stc, value=(
            f'=IF(NOT(ISNUMBER({vref})),"No data",IF($B${br}="higher",'
            f'IF({vref}>=$C${br},"Green",IF({vref}>=$D${br},"Amber","Red")),'
            f'IF({vref}<=$C${br},"Green",IF({vref}<=$D${br},"Amber","Red"))))'))
        ws.cell(row=r, column=vc).number_format = PCT if b.kpi in pct_kpis else (COVER if b.kpi == "interest_cover" else
TIMES)
        status_cells.append(f"{get_column_letter(stc)}{r}")
    status_cells_all += status_cells
    scored = "+".join(f'({s}<>"No data")' for s in status_cells)
    points = "+".join(f'IF({s}="Green",2,IF({s}="Amber",1,0))' for s in status_cells)
    ws.cell(row=r, column=sc, value=f"={scored}")
    ws.cell(row=r, column=sc + 1, value=f'=IF({get_column_letter(sc)}{r}>0,({points})/(2*{get_column_letter(sc)}{r}),"n/
a")')
    ws.cell(row=r, column=sc + 1).number_format = "0%"
    for c in range(1, total_cols + 1):
        cell = ws.cell(row=r, column=c)
        cell.border = BOX
        if c > 1:
            cell.font = F_BODY
            cell.alignment = CENTER
    ws.cell(row=r, column=1).alignment = Alignment(vertical="center", indent=1)
    ws.cell(row=r, column=sc + 1).font = Font(name=FONT, size=11, bold=True, color=INK)
    ws.row_dimensions[r].height = 30
end = top + 1 + len(kpi_order)
for col_i in range(4, 4 + 2 * n_k, 2):
    L = get_column_letter(col_i)
    rng_s = f"{L}{top + 2}:{L}{end}"
    for word, fill, colour in (("Green", GOOD_FILL, "1E7B3A"), ("Amber", WARN_FILL, "B26A00"),
                               ("Red", BAD_FILL, "B42318"), ("No data", "EEF1F6", "5B6577")):
        ws.conditional_formatting.add(rng_s, CellIsRule(operator="equal", formula=[f'"{word}"'],
                                      fill=PatternFill("solid", fgColor=fill), font=Font(name=FONT, bold=True,
color=colour)))
score_rng = f"{get_column_letter(sc + 1)}{top + 2}:{get_column_letter(sc + 1)}{end}"
ws.conditional_formatting.add(score_rng, CellIsRule(operator="greaterThanOrEqual", formula=["0.8"],
fill=PatternFill("solid", fgColor=GOOD_FILL)))
ws.conditional_formatting.add(score_rng, CellIsRule(operator="lessThan", formula=["0.5"], fill=PatternFill("solid",
fgColor=BAD_FILL)))
notes = [
    "Screening score = points ÷ maximum possible points, where green = 2, amber = 1, red = 0. KPIs with no data are left
out, not scored as red.",
    "Polar Capital scores 100% on only 5 of 7 KPIs (no operating profit line or separate lease figure), so its score rests
on less evidence.",
    "Equity change compares latest total equity with the base year three years earlier. Interest coverage is 'No data'
where a company reports no interest cost.",
    "The score is a quick screen to decide where to look first. It is not a credit rating and does not show that any
company is safe or at risk.",
]
for j, t in enumerate(notes):
    c = ws.cell(row=end + 2 + j, column=1, value=t)
    c.font = F_NOTE
    ws.merge_cells(start_row=end + 2 + j, start_column=1, end_row=end + 2 + j, end_column=total_cols)
set_widths(ws, {"A": 32, "B": 11, **{get_column_letter(c): 10.5 for c in range(3, total_cols + 1)}})
ws.column_dimensions["E"].width = 10.5
ws.freeze_panes = ws.cell(row=top + 2, column=3)
page_setup(ws)

# ============================================================== README sheet
import re

from openpyxl.drawing.image import Image as XLImage
from openpyxl.worksheet.hyperlink import Hyperlink

SHEET_NAMES = set(wb.sheetnames)
LINK_RE = re.compile(r"\[([^\]]+)\]\(([^)]+)\)")


def plain(text: str) -> str:
    text = LINK_RE.sub(lambda m: m.group(1), text)
    return text.replace("**", "").replace("`", "").replace("*", "").strip()


def first_link(text: str):
    m = LINK_RE.search(text)
    return m.group(2) if m else None


def bold_names(text: str) -> list[str]:
    return re.findall(r"\*\*([^*]+)\*\*", text)


from config import github_link


def resolve(target: str) -> str | None:
    """Turn a README link into a link that works from a downloaded workbook.

    Files and folders in the project point to their page in the GitHub
```

```
    repository, so the link still works when the workbook is downloaded on
    its own.
    """
    if target.startswith("http"):
        return target
    if target.startswith("#"):
        return None                       # section anchor, handled separately
    return github_link(target)


def slug(heading: str) -> str:
    s = re.sub(r"[^\w\s-]", "", heading.lower()).strip()
    return re.sub(r"\s+", "-", s)


ws = ws_readme
md = (ROOT / "README.md").read_text(encoding="utf-8").split("\n")
WIDTH_COLS = 8
COL_W = 19
set_widths(ws, {get_column_letter(c): COL_W for c in range(1, WIDTH_COLS + 1)})
CHARS_PER_ROW = int(COL_W * WIDTH_COLS * 0.92)
anchors, pending = {}, []
embedded = set()
row = 1
i = 0
in_code = False


def text_row(r, text, font=F_BODY, fill=None, indent=0, link=None, height=None):
    cell = ws.cell(row=r, column=1, value=text)
    ws.merge_cells(start_row=r, start_column=1, end_row=r, end_column=WIDTH_COLS)
    cell.font = font
    cell.alignment = Alignment(wrap_text=True, vertical="top", indent=indent)
    if fill:
        for c in range(1, WIDTH_COLS + 1):
            ws.cell(row=r, column=c).fill = fill
    lines = max(1, -(-len(text) // CHARS_PER_ROW))
    ws.row_dimensions[r].height = height or max(15, 14 * lines + 3)
    if link:
        target = resolve(link)
        if target:
            cell.hyperlink = target
            if font is F_BODY:
                cell.font = Font(name=FONT, size=10, color="1155CC", underline="single")
        elif link.startswith("#"):
            pending.append((cell, link[1:]))
    return cell


while i < len(md):
    line = md[i]
    s = line.strip()
    if s.startswith("```"):
        in_code = not in_code
        if not in_code:
            row += 1          # gap after a code block
        i += 1
        continue
    if in_code:
        text_row(row, line if line else " ", font=Font(name="Consolas", size=9, color=INK),
                 fill=PatternFill("solid", fgColor="EEF1F6"), indent=1, height=15)
        row += 1
        i += 1
        continue
    if not s or s == "---":
        row += 0 if not s else 1
        i += 1
        continue
    if s.startswith("# "):
        title_band(ws, plain(s[2:]), "Project guide. The same text as README.md in the project folder.", WIDTH_COLS)
        row = 4
    elif s.startswith("## "):
        row += 1
        head = plain(s[3:])
        text_row(row, head, font=Font(name=FONT, size=12, bold=True, color=WHITE), fill=FILL_INK, indent=1, height=22)
        anchors[slug(head)] = row
        row += 1
    elif s.startswith("!["):
        img_path = ROOT / first_link(s)
        if img_path.name.startswith("excel_") or img_path in embedded:
            # screenshots of this workbook are not embedded in it; link instead
            text_row(row, f"Image: {first_link(s)}", link=first_link(s))
            row += 1
        elif img_path.exists():
            embedded.add(img_path)
            pic = XLImage(str(img_path))
            scale = (COL_W * WIDTH_COLS * 7.0) / pic.width
            pic.width, pic.height = int(pic.width * scale), int(pic.height * scale)
            ws.add_image(pic, f"A{row}")
            rows_needed = int(pic.height / 20) + 1
            for r in range(row, row + rows_needed):
                ws.row_dimensions[r].height = 15
            row += rows_needed + 1
    elif s.startswith("|"):
        table = []
        while i < len(md) and md[i].strip().startswith("|"):
            cells = [c.strip() for c in md[i].strip().strip("|").split("|")]
            if not all(re.fullmatch(r":?-{2,}:?", c) for c in cells):
                table.append(cells)
            i += 1
```

```
        ncols = len(table[0])
        if ncols > WIDTH_COLS:
            spans = [1] * ncols
        else:
            spans = [WIDTH_COLS // ncols] * ncols
            spans[-1] += WIDTH_COLS - sum(spans)
        if ncols == 2:
            spans = [3, 5]
        for t_i, cells in enumerate(table):
            col = 1
            longest = 1
            for c_i, raw_txt in enumerate(cells):
                txt = plain(raw_txt)
                span = spans[c_i]
                cell = ws.cell(row=row, column=col, value=txt)
                if span > 1:
                    ws.merge_cells(start_row=row, start_column=col, end_row=row, end_column=col + span - 1)
                for cc in range(col, col + span):
                    ws.cell(row=row, column=cc).border = BOX
                cell.alignment = Alignment(wrap_text=True, vertical="top")
                if t_i == 0:
                    cell.font = F_H
                    for cc in range(col, col + span):
                        ws.cell(row=row, column=cc).fill = FILL_INK
                else:
                    cell.font = F_BOLD if c_i == 0 else F_BODY
                    name = txt if txt in SHEET_NAMES else None
                    link = first_link(raw_txt)
                    if name:
                        cell.hyperlink = Hyperlink(ref=cell.coordinate, location=f"'{name}'!A1")
                        cell.font = Font(name=FONT, size=10, bold=True, color="1155CC", underline="single")
                    elif link and resolve(link):
                        cell.hyperlink = resolve(link)
                        cell.font = Font(name=FONT, size=10, color="1155CC", underline="single", bold=(c_i == 0))
                    company = next((co for co in companies["company"] if co == txt or SHORT[co] == txt), None)
                    if company and c_i == 0:
                        cell.font = Font(name=FONT, size=10, bold=True, color=BRAND[company])
                longest = max(longest, -(-len(txt) // int(COL_W * span * 1.05)))
                col += span
            ws.row_dimensions[row].height = max(18, 14 * longest + 4)
            row += 1
        row += 1
        continue
    elif s.startswith(">"):
        txt = plain(s.lstrip("> "))
        if txt:
            text_row(row, txt, font=Font(name=FONT, size=10, italic=True, color=INK),
                     fill=PatternFill("solid", fgColor="EEF1F6"), indent=1)
            row += 1
    elif re.match(r"^(- |\d+\. )", s):
        marker = "•" if s.startswith("- ") else s.split(" ", 1)[0]
        body = s[2:] if s.startswith("- ") else s.split(" ", 1)[1]
        text_row(row, f"{marker}  {plain(body)}", indent=1, link=first_link(body))
        row += 1
    else:
        text_row(row, plain(s), font=F_BOLD if s.startswith("**") and s.endswith("**") else F_BODY,
                 link=first_link(s) if s.count("](") == 1 else None)
        row += 1
    i += 1

for cell, target in pending:
    if target in anchors:
        cell.hyperlink = Hyperlink(ref=cell.coordinate, location=f"README!A{anchors[target]}")
        cell.font = Font(name=FONT, size=10, color="1155CC", underline="single")
ws.sheet_view.showGridLines = False
ws.freeze_panes = "A3"
page_setup(ws, landscape=False)

# ---- workbook-wide defaults
for sheet in wb.worksheets:
    sheet.sheet_properties.tabColor = INK if sheet.title == "Dashboard" else "8A94A6"
wb.active = 0
wb.calculation.fullCalcOnLoad = True   # Excel recalculates every formula on open
OUT.mkdir(exist_ok=True)
wb.save(FILE)
print(f"Saved {FILE}")

# ---- optional: one file per sheet, used by src/make_screenshots.py
import sys

if "--screens" in sys.argv:
    shots = OUT / "_screens"
    shots.mkdir(exist_ok=True)
    for target in ("Dashboard", "KPI_Scorecard", "Ratios", "Project_Story", "Data_Log"):
        for sheet in wb.worksheets:
            sheet.sheet_state = "visible" if sheet.title == target else "hidden"
        wb.active = wb.sheetnames.index(target)
        page = wb[target]
        old = page.page_setup.fitToHeight
        page.page_setup.fitToHeight = 1          # whole sheet on one page for the image
        wb.save(shots / f"{target}.xlsx")
        page.page_setup.fitToHeight = old
    print(f"Saved per-sheet copies to {shots}")
```

A4. Python: dashboard image (src/make_dashboard.py)

```
"""
Draw the dashboard image used in the README (images/dashboard.png).
It shows the same numbers as the Dashboard sheet in the workbook.
```

```
Run after src/analysis.py:
    python src/make_dashboard.py
"""

from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.patches import FancyBboxPatch

ROOT = Path(__file__).resolve().parents[1]
r = pd.read_csv(ROOT / "outputs" / "ratios.csv")
co = pd.read_csv(ROOT / "data" / "companies.csv")
brand = {row.short_name: "#" + row.brand_colour for row in co.itertuples()}
r = r[r["in_analysis"] == 1].copy()
r["pos"] = r.groupby("company")["fy_end"].rank().astype(int)  # 1,2,3 = oldest..latest
latest = r[r["pos"] == 3].sort_values("pbt_margin", ascending=False)
order = latest["short_name"].tolist()

INK, MUTED, GRID = "#1F2A44", "#5B6577", "#E3E7EE"
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 10, "axes.edgecolor": GRID,
                     "axes.labelcolor": MUTED, "xtick.color": MUTED, "ytick.color": MUTED})

fig = plt.figure(figsize=(16, 10), dpi=150, facecolor="white")
fig.patches.append(FancyBboxPatch((0, 0.915), 1, 0.085, transform=fig.transFigure,
                                  boxstyle="square,pad=0", fc=INK, ec="none"))
fig.text(0.02, 0.962, "UK Asset & Wealth Managers: Financial Health Review", color="white",
         fontsize=20, weight="bold", va="center")
fig.text(0.02, 0.932, "Latest audited year per company · £ millions · group accounts listed on Companies House · data
accessed 4-5 Nov 2026",
         color="#DCE3EE", fontsize=10.5, va="center")

# ---- scorecard table
ax_t = fig.add_axes([0.02, 0.56, 0.96, 0.34])
ax_t.axis("off")
cols = ["Company", "Latest year", "Revenue £m", "Revenue growth", "Profit margin (PBT)",
        "Current ratio", "Debt-to-equity", "Interest cover"]
xs = [0.0, 0.20, 0.30, 0.41, 0.53, 0.65, 0.76, 0.87]
ax_t.add_patch(plt.Rectangle((0, 0.86), 1, 0.14, color=INK, transform=ax_t.transAxes))
for x, c in zip(xs, cols):
    ax_t.text(x + (0.01 if x == 0 else 0.05), 0.93, c, color="white", weight="bold",
              fontsize=10.5, va="center", ha="left" if x == 0 else "center", transform=ax_t.transAxes)

def fmt_pct(v):
    return "n/a" if pd.isna(v) else f"{v:+.1%}" if v != 0 else "0.0%"

row_h = 0.86 / len(order)
for i, name in enumerate(order):
    d = latest[latest["short_name"] == name].iloc[0]
    y = 0.86 - (i + 1) * row_h
    if i % 2 == 0:
        ax_t.add_patch(plt.Rectangle((0, y), 1, row_h, color="#F5F7FA", transform=ax_t.transAxes))
    ax_t.add_patch(plt.Rectangle((0, y + 0.01), 0.19, row_h - 0.02, color=brand[name], transform=ax_t.transAxes))
    ax_t.text(0.01, y + row_h / 2, name, color="white", weight="bold", fontsize=11.5, va="center",
transform=ax_t.transAxes)
    cells = [d["fy_label"], f"{d['revenue_m']:,.1f}", fmt_pct(d["revenue_growth"]),
             f"{d['pbt_margin']:.1%}", f"{d['current_ratio']:.2f}x",
             "n/a" if pd.isna(d["debt_to_equity"]) else f"{d['debt_to_equity']:.2f}x",
             "n/a" if pd.isna(d["interest_cover"]) else f"{d['interest_cover']:,.0f}x"]
    colours = [INK, INK,
               "#B42318" if d["revenue_growth"] < 0 else "#1E7B3A",
               "#1E7B3A" if d["pbt_margin"] >= 0.25 else ("#B42318" if d["pbt_margin"] < 0.05 else INK),
               "#B26A00" if d["current_ratio"] < 1.5 else INK, INK, INK]
    for x, txt, colr in zip(xs[1:], cells, colours):
        ax_t.text(x + 0.05, y + row_h / 2, txt, color=colr, fontsize=11.5, va="center", ha="center",
                  weight="bold" if colr != INK else "normal", transform=ax_t.transAxes)

# ---- charts
labels = ["Two years before", "One year before", "Latest year"]

def trend(ax, col, title, pct=True):
    for name in order:
        s = r[r["short_name"] == name].sort_values("pos")[col].values
        ax.plot(labels, s, marker="o", lw=2.4, ms=6, color=brand[name], label=name)
    ax.set_title(title, loc="left", fontsize=12.5, weight="bold", color=INK, pad=8)
    ax.grid(axis="y", color=GRID)
    ax.axhline(0, color="#9AA3B2", lw=0.8)
    if pct:
        ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)

ax1 = fig.add_axes([0.04, 0.10, 0.28, 0.38])
trend(ax1, "pbt_margin", "Profit before tax margin")
ax2 = fig.add_axes([0.37, 0.10, 0.28, 0.38])
trend(ax2, "revenue_growth", "Revenue growth")
ax3 = fig.add_axes([0.72, 0.10, 0.26, 0.38])
vals = latest.set_index("short_name").loc[order, "current_ratio"]
ax3.barh(order[::-1], vals[::-1], color=[brand[n] for n in order[::-1]], height=0.6)
for y, v in enumerate(vals[::-1]):
    ax3.text(v + 0.12, y, f"{v:.2f}x", va="center", fontsize=10, color=INK)
ax3.set_title("Current ratio, latest year", loc="left", fontsize=12.5, weight="bold", color=INK, pad=8)
ax3.axvline(1, color="#9AA3B2", lw=0.8, ls="--")
ax3.set_xlim(0, vals.max() * 1.18)
for side in ("top", "right"):
    ax3.spines[side].set_visible(False)

handles, lbls = ax1.get_legend_handles_labels()
fig.legend(handles, lbls, loc="lower center", ncol=6, frameon=False, bbox_to_anchor=(0.36, 0.005), fontsize=10)
```

```
fig.text(0.98, 0.015, "Ratios point to questions to ask, not verdicts. Sources: see Sources sheet.",
         ha="right", fontsize=9, color=MUTED, style="italic")

out = ROOT / "images" / "dashboard.png"
out.parent.mkdir(exist_ok=True)
fig.savefig(out, facecolor="white")
print(f"Saved {out}")
```

A5. Python: workbook screenshots (src/make_screenshots.py)

```
"""
Make the workbook screenshots shown in the README (images/excel_*.png).

Needs LibreOffice (soffice) and poppler (pdftoppm) on the PATH.
Run after src/analysis.py:
    python src/make_screenshots.py
"""

import shutil
import subprocess
from pathlib import Path

from PIL import Image, ImageChops

ROOT = Path(__file__).resolve().parents[1]
SHOTS = ROOT / "outputs" / "_screens"
IMAGES = ROOT / "images"
SHEETS = {
    "Dashboard": "excel_dashboard.png",
    "KPI_Scorecard": "excel_kpi_scorecard.png",
    "Ratios": "excel_ratios.png",
    "Project_Story": "excel_project_story.png",
    "Data_Log": "excel_data_log.png",
}


def trim(img: Image.Image, pad: int = 24) -> Image.Image:
    """Cut the blank page margins, keeping a small white border."""
    bg = Image.new(img.mode, img.size, (255, 255, 255))
    box = ImageChops.difference(img, bg).getbbox()
    if not box:
        return img
    l, t, r, b = box
    return img.crop((max(l - pad, 0), max(t - pad, 0), min(r + pad, img.width), min(b + pad, img.height)))


def main() -> None:
    subprocess.run(["python", str(ROOT / "src" / "build_workbook.py"), "--screens"], check=True)
    IMAGES.mkdir(exist_ok=True)
    for sheet, name in SHEETS.items():
        xlsx = SHOTS / f"{sheet}.xlsx"
        subprocess.run(["soffice", "--headless", "--convert-to", "pdf", "--outdir", str(SHOTS), str(xlsx)],
                       check=True, capture_output=True, timeout=180)
        stem = SHOTS / sheet
        subprocess.run(["pdftoppm", "-r", "170", "-png", "-f", "1", "-l", "1", "-singlefile",
                        str(stem.with_suffix(".pdf")), str(stem)], check=True)
        img = trim(Image.open(stem.with_suffix(".png")).convert("RGB"))
        img.save(IMAGES / name, optimize=True)
        print(f"Saved images/{name} ({img.width}x{img.height})")
    shutil.rmtree(SHOTS)


if __name__ == "__main__":
    main()
```

A6. Python: this report (src/build_report.py)

```
"""
Write the report: report/UK_Financial_Health_Review_Report.docx and .pdf
The main report is about three pages, followed by a tools section and an
appendix holding all project code (SQL and Python).
Run: python src/build_report.py
"""

import subprocess
from pathlib import Path

from docx import Document
from docx.enum.table import WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "report" / "UK_Financial_Health_Review_Report.docx"
INK = RGBColor(0x1F, 0x2A, 0x44)
MUTED = RGBColor(0x5B, 0x65, 0x77)

doc = Document()
sec = doc.sections[0]
sec.page_height, sec.page_width = Cm(29.7), Cm(21.0)
for side in ("left_margin", "right_margin"):
    setattr(sec, side, Cm(2.0))
sec.top_margin, sec.bottom_margin = Cm(1.8), Cm(1.8)

base = doc.styles["Normal"]
base.font.name, base.font.size = "Arial", Pt(10)
```

```
base.element.rPr.rFonts.set(qn("w:eastAsia"), "Arial")
base.paragraph_format.space_after = Pt(5)
base.paragraph_format.line_spacing = 1.12
for name, size in (("Heading 1", 13), ("Heading 2", 11)):
    st = doc.styles[name]
    st.font.name, st.font.size, st.font.bold = "Arial", Pt(size), True
    st.font.color.rgb = INK
    st.element.rPr.rFonts.set(qn("w:eastAsia"), "Arial")
    st.paragraph_format.space_before, st.paragraph_format.space_after = Pt(10), Pt(4)


def para(text, bold_lead=None, style=None, size=None, colour=None, italic=False):
    p = doc.add_paragraph(style=style)
    if bold_lead:
        r = p.add_run(bold_lead)
        r.bold = True
    r = p.add_run(text)
    r.italic = italic
    if size:
        r.font.size = Pt(size)
    if colour:
        r.font.color.rgb = colour
    return p


def bullet(text, bold_lead=None):
    p = para(text, bold_lead, style="List Bullet")
    p.paragraph_format.space_after = Pt(3)
    return p


def shade(cell, hex_fill):
    tcPr = cell._tc.get_or_add_tcPr()
    shd = OxmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), hex_fill)
    tcPr.append(shd)


# ---------------------------------------------------------------- title
t = doc.add_paragraph()
r = t.add_run("UK Company Financial Health Review")
r.bold, r.font.size, r.font.color.rgb = True, Pt(18), INK
para("UK-listed asset and wealth managers · data from audited group accounts · accessed 4-5 November 2026",
     size=9.5, colour=MUTED)

doc.add_heading("1. The question", level=1)
para("Which companies in one UK sector look financially stronger, and what should a lender or investor "
     "look into before relying on them? I chose six UK-listed asset and wealth managers: AJ Bell, Ashmore, "
     "Jupiter, Liontrust, Polar Capital and Premier Miton. These firms report their balance sheets in current "
     "and non-current items, so all five ratios can be worked out. Banks and insurers do not report this way, "
     "which is why I did not use them.")

doc.add_heading("2. Data sources", level=1)
para("Each company was identified by its Companies House number (04503206, 03675683, 06150195, 02954692, "
     "04235369 and 06306664). The Companies House pages confirmed the company names, the year ends and, for "
     "most of them, the group accounts filed. The accounts PDFs held at Companies House could not be opened "
     "in this working environment. I therefore took the figures from the same audited group statements that "
     "each company publishes: its annual report, or its audited results announcement where the full report "
     "could not be read. Every figure in the workbook lists its source document and web link.")
para("For each company I used two documents, each showing two years. That gives four years of figures: "
     "three years for the ratios, plus an earlier year as the starting point for revenue growth. Ashmore's "
     "results for the year to June 2026 have been published, but I could not retrieve the full statements. "
     "Its analysis therefore stops at FY2025.")

doc.add_heading("3. Method", level=1)
para("The figures were typed in exactly as printed. Amounts in £000 were then converted to £ millions. "
     "Blank fields were left blank rather than estimated. Every ratio in the workbook is a live Excel "
     "formula, and the same calculations are repeated in Python and checked with SQL queries.")
for lead, text in [
    ("Revenue growth", " = this year's revenue ÷ last year's revenue − 1."),
    ("Profit margin", " = profit before tax ÷ revenue. I used profit before tax because Polar Capital does not "
                      "report an operating profit line, so it is the one profit measure all six share. "
                      "Operating margin is shown alongside as a check."),
    ("Current ratio", " = current assets ÷ current liabilities."),
    ("Debt-to-equity", " = (borrowings + lease liabilities) ÷ total equity."),
    ("Interest coverage", " = (profit before tax + finance costs) ÷ finance costs."),
]:
    bullet(text, lead)

doc.add_heading("4. Findings", level=1)
para("The table shows each company's latest year. The workbook sets out all three years.")
rows = [
    ("Company", "Year", "Revenue £m", "Growth", "PBT margin", "Current ratio", "Debt / equity"),
    ("AJ Bell", "FY2025", "317.8", "+18.0%", "43.4%", "3.65x", "0.06x"),
    ("Polar Capital", "FY2026", "264.3", "+19.0%", "29.1%", "2.53x", "n/a"),
    ("Jupiter", "FY2025", "465.7", "+15.7%", "28.3%", "2.46x", "0.04x"),
    ("Ashmore", "FY2025", "144.4", "−23.6%", "75.2%", "7.01x", "0.01x"),
    ("Liontrust", "FY2026", "134.4", "−20.9%", "10.7%", "1.34x", "0.03x"),
    ("Premier Miton", "FY2025", "63.3", "−1.1%", "3.7%", "1.27x", "0.02x"),
]
brand = {"AJ Bell": "5B2C83", "Polar Capital": "2F80C0", "Jupiter": "00796B", "Ashmore": "14233C",
         "Liontrust": "E07B00", "Premier Miton": "C2185B"}
tbl = doc.add_table(rows=len(rows), cols=len(rows[0]))
tbl.alignment = WD_TABLE_ALIGNMENT.CENTER
tbl.style = "Table Grid"
for i, row in enumerate(rows):
    for j, val in enumerate(row):
        cell = tbl.cell(i, j)
```

```
        cell.text = ""
        p = cell.paragraphs[0]
        p.paragraph_format.space_after = Pt(0)
        run = p.add_run(val)
        run.font.size = Pt(9)
        if i == 0:
            run.bold = True
            run.font.color.rgb = RGBColor(0xFF, 0xFF, 0xFF)
            shade(cell, "1F2A44")
        elif j == 0:
            run.bold = True
            run.font.color.rgb = RGBColor(0xFF, 0xFF, 0xFF)
            shade(cell, brand[val])
        if j > 0:
            p.alignment = WD_ALIGN_PARAGRAPH.CENTER
para("Polar Capital's lease liabilities are not shown separately, so its debt-to-equity is left as n/a. "
     "Interest coverage is in the workbook. It is very high wherever the only interest cost is on office "
     "leases.", size=8.5, colour=MUTED, italic=True)

doc.add_heading("KPI screening", level=2)
para("The five ratios, plus operating margin and the change in equity over three years, were turned into seven "
     "KPIs, each rated green, amber or red against fixed bands (for example, a profit margin of 25% or more is "
     "green, under 10% is red). Green scores 2 points, amber 1 and red 0, and KPIs with no data are left out. "
     "On this screen AJ Bell and Jupiter score 100% on all seven KPIs, and Polar Capital 100% on the five it "
     "reports. Ashmore scores 67%, green on its balance sheet but red on revenue growth and equity change. "
     "Liontrust scores 43% and Premier Miton 17%. The bands are my own reading aids, and the score shows where "
     "to look first; it is not a rating.")

doc.add_heading("Stronger on these measures", level=2)
bullet(" Revenue rose from £218.2m in FY2023 to £317.8m in FY2025, growing 33.2%, 23.5% and 18.0% a year. "
       "The profit margin edged up from 40.2% to 43.4%. The current ratio held at about 3.6x, and its only "
       "debt is £12.1m of leases against £217.5m of equity.", "AJ Bell.")
bullet(" Revenue grew in each of the three years, by 6.7%, 13.9% and 19.0%, reaching £264.3m in FY2026. "
       "The margin stayed between 23% and 29%, and the current ratio was steady at about 2.5x. Its only "
       "interest cost is about £0.2m on leases.", "Polar Capital.")
bullet(" The FY2023 margin was only 2.3% because of a £76.2m goodwill write-down. It then recovered to 21.9% "
       "and 28.3%. Jupiter repaid its £50m subordinated notes in April 2025, so debt-to-equity fell from "
       "0.12x to 0.04x. The current ratio improved from 2.09x to 2.46x.", "Jupiter.")

doc.add_heading("Mixed", level=2)
bullet(" Ashmore has the largest liquidity cushion (current ratio 7.0x to 10.7x) and no debt. However, "
       "revenue fell from £193.2m to £144.4m over the three years, and equity fell from £913.0m to £790.8m. "
       "Its 75.2% margin is flattered by £51.1m of finance income, which is 47% of profit before tax. "
       "Operating margin, at 30% to 40%, gives a better view of the fee business.", "Ashmore.")

doc.add_heading("Weaker on these measures", level=2)
bullet(" Revenue fell in each of the three years, from £243.3m in FY2023 to £134.4m in FY2026, a drop of 45%. "
       "It made a small loss before tax in FY2024. Equity fell from £221.0m to £107.9m. Its current ratio "
       "(1.34x) is the second lowest in the group.", "Liontrust.")
bullet(" Revenue fell from £90.2m in FY2022 to £63.3m in FY2025, about 30% lower. The margin narrowed each "
       "year, from 7.9% to 4.9% and then 3.7%. It has no bank debt, but has the lowest current ratio at "
       "1.27x.", "Premier Miton.")

doc.add_heading("5. Risks a lender or investor should examine", level=1)
bullet(" Fees are charged on assets under management, so a market fall or client outflows reduce revenue "
       "quickly. This shows clearly at Liontrust, Premier Miton and Ashmore. Ask for the latest flows and "
       "assets under management, not just the accounts.", "Market and outflow risk.")
bullet(" Ashmore's profit depends heavily on returns from its own investments, and Jupiter's FY2023 result "
       "was hit by a one-off write-down. Check how much profit comes from the core fee business.",
       "Quality of profit.")
bullet(" For AJ Bell and other platforms, part of revenue is interest earned on client cash. A cut in UK "
       "interest rates could reduce it. Ask what share of revenue this is.", "Interest-rate sensitivity.")
bullet(" Money moving between investors and funds can sit in receivables and payables. These amounts are "
       "large and largely offset each other, which pulls the current ratio towards 1x. Ask for the "
       "breakdown before treating 1.3x as a warning sign.", "What the current ratio contains.")
bullet(" Liontrust's equity halved in three years as dividends and buybacks ran ahead of profit. Check "
       "dividend cover and the regulatory capital surplus reported in each firm's Pillar 3 or ICARA "
       "disclosures.", "Shrinking equity and capital needs.")
bullet(" Most of these firms owe little apart from leases, so debt ratios say little about them. A lender "
       "should look at covenants, guarantees, seed-capital commitments and any defined benefit pension "
       "scheme.", "Hidden obligations.")

doc.add_heading("6. Limits of the data", level=1)
bullet(" The year ends are March, June, September and December, so the latest years run from June 2025 to "
       "March 2026. Market conditions were not the same across these dates.", "Different dates.")
bullet(" The figures come from the audited statements each company published, not from Companies House "
       "PDFs, which could not be read here. AJ Bell's figures come from its audited results "
       "announcements. The Companies House pages for AJ Bell and Ashmore came from older cached copies, "
       "and Jupiter's page could not be opened.", "Source route.")
bullet(" Polar Capital does not report an operating profit line or separate lease liabilities. Ashmore "
       "reports no finance cost line. These cells are marked n/a, not estimated.", "Missing figures.")
bullet(" Each firm labels its lines differently ('net revenue', 'gross profit', 'net income'). Some prior "
       "years were re-presented. Each case is listed in the workbook's Data_Log sheet.",
       "Reporting differences.")
bullet(" These are listed groups, but the same approach applied to private companies is harder. Small "
       "companies can file abridged accounts with no profit and loss account, so growth and margin cannot "
       "be calculated at all.", "Private companies.")

doc.add_heading("7. Conclusion", level=1)
para("On these five measures, AJ Bell, Polar Capital and Jupiter look the strongest. They show growing revenue, "
     "healthy margins, comfortable current ratios and little or no borrowing. Ashmore has a very strong "
     "balance sheet but shrinking fee income. Liontrust and Premier Miton show falling revenue and thin "
     "margins, and their questions are about the trend more than about solvency today. None of this proves "
     "that any firm is safe or likely to fail. The ratios show where a lender or investor should ask "
     "further questions before deciding.")

import csv
import sys

sys.path.insert(0, str(ROOT / "src"))
```

```
from config import REPO_URL, github_link  # noqa: E402


def add_link(paragraph, url: str, text: str, size: float = 8.5):
    """Insert a clickable hyperlink run into a paragraph."""
    r_id = paragraph.part.relate_to(url, "http://schemas.openxmlformats.org/officeDocument/2006/relationships/hyperlink",
                                    is_external=True)
    link = OxmlElement("w:hyperlink")
    link.set(qn("r:id"), r_id)
    run = OxmlElement("w:r")
    rpr = OxmlElement("w:rPr")
    for tag, attr in (("w:color", "1155CC"), ("w:u", "single"), ("w:sz", str(int(size * 2)))):
        el = OxmlElement(tag)
        el.set(qn("w:val"), attr)
        rpr.append(el)
    fonts = OxmlElement("w:rFonts")
    fonts.set(qn("w:ascii"), "Arial")
    fonts.set(qn("w:hAnsi"), "Arial")
    rpr.append(fonts)
    run.append(rpr)
    t = OxmlElement("w:t")
    t.text = text
    t.set(qn("xml:space"), "preserve")
    run.append(t)
    link.append(run)
    paragraph._p.append(link)


doc.add_heading("8. Sources and project files", level=1)
para("All links were checked on 5 November 2026. Figures were accessed on 4-5 November 2026.", size=8.5, colour=MUTED)
with open(ROOT / "data" / "sources.csv", newline="", encoding="utf-8") as fh:
    for src in csv.DictReader(fh):
        p = doc.add_paragraph()
        p.paragraph_format.space_after = Pt(2)
        lead = p.add_run(f"{src['source_id']}  {src['company']}: {src['document']}. ")
        lead.font.size = Pt(8.5)
        add_link(p, src["url"], "Open source")
with open(ROOT / "data" / "companies.csv", newline="", encoding="utf-8") as fh:
    p = doc.add_paragraph()
    p.paragraph_format.space_before = Pt(4)
    r = p.add_run("Companies House pages: ")
    r.font.size, r.bold = Pt(8.5), True
    for k, co in enumerate(csv.DictReader(fh)):
        if k:
            sep = p.add_run(" · ")
            sep.font.size = Pt(8.5)
        add_link(p, co["companies_house_link"], f"{co['short_name']} ({co['company_number']})")
p = doc.add_paragraph()
r = p.add_run("Project repository (workbook, data, code and dashboard images): ")
r.font.size, r.bold = Pt(8.5), True
add_link(p, REPO_URL, REPO_URL)
p = doc.add_paragraph()
r = p.add_run("Direct files: ")
r.font.size, r.bold = Pt(8.5), True
for k, (label, path) in enumerate([("Excel workbook", "outputs/UK_Financial_Health_Review.xlsx"),
                                   ("Dashboard image", "images/dashboard.png"),
                                   ("KPI scorecard", "outputs/kpi_scorecard.csv"),
                                   ("SQL queries", "sql/queries.sql"),
                                   ("Python code", "src")]):
    if k:
        sep = p.add_run(" · ")
        sep.font.size = Pt(8.5)
    add_link(p, github_link(path), label)

doc.add_heading("9. Tools used", level=1)
tools = [
    ("Tool", "What it was used for"),
    ("Companies House (find-and-update service)", "Confirming company names and numbers, SIC codes, year ends and filed
accounts records."),
    ("Company annual reports and audited results", "Source of every financial figure (see the Sources sheet for links and
access dates)."),
    ("Python 3 with pandas", "Importing the raw figures, converting units, quality checks, ratio and KPI calculations
(src/analysis.py)."),
    ("SQL (SQLite via Python's sqlite3)", "Checks on the results: latest-year snapshot, three-year averages, revenue
declines, thin margins, missing inputs (sql/queries.sql)."),
    ("Microsoft Excel format (openpyxl)", "Workbook with live formulas, conditional formatting and native charts. The
Dashboard sheet is the BI layer of the project."),
    ("matplotlib and Pillow", "Summary dashboard image and trimmed workbook screenshots for the README
(src/make_dashboard.py, src/make_screenshots.py)."),
    ("GitHub", "Hosting the project; links in the workbook and report point to the repository so they work after download
(src/config.py, src/check_links.py)."),
    ("python-docx and LibreOffice", "Writing this report and converting it to PDF; LibreOffice also recalculated and
checked every workbook formula."),
    ("BI platform", "No separate BI tool (such as Power BI or Tableau) was used. outputs/ratios.csv and
outputs/financials.db can be loaded into one directly."),
]
tt = doc.add_table(rows=len(tools), cols=2)
tt.style = "Table Grid"
for i, (a, b) in enumerate(tools):
    for j, val in enumerate((a, b)):
        cell = tt.cell(i, j)
        cell.text = ""
        p = cell.paragraphs[0]
        p.paragraph_format.space_after = Pt(0)
        run = p.add_run(val)
        run.font.size = Pt(9)
        if i == 0:
            run.bold = True
            run.font.color.rgb = RGBColor(0xFF, 0xFF, 0xFF)
            shade(cell, "1F2A44")
```

```
        elif j == 0:
            run.bold = True
for row in tt.rows:
    row.cells[0].width, row.cells[1].width = Cm(5.5), Cm(11.5)

# ---------------------------------------------------------------- appendix: code
CODE_FILES = [
    ("A1. SQL checks", "sql/queries.sql"),
    ("A2. Python: import, cleaning, quality checks, ratios and SQL", "src/analysis.py"),
    ("A3. Python: Excel workbook and dashboard (BI layer)", "src/build_workbook.py"),
    ("A4. Python: dashboard image", "src/make_dashboard.py"),
    ("A5. Python: workbook screenshots", "src/make_screenshots.py"),
    ("A6. Python: this report", "src/build_report.py"),
    ("A7. Python: shared settings (GitHub address)", "src/config.py"),
    ("A8. Python: link checker", "src/check_links.py"),
]
doc.add_page_break()
doc.add_heading("Appendix: project code", level=1)
para("The complete code used for the analysis, as stored in the project folder. Run the scripts in this order: "
     "analysis.py, build_workbook.py, make_dashboard.py, make_screenshots.py, build_report.py, then check_links.py. The
KPI bands are in data/kpi_thresholds.csv. No API keys or passwords are included; "
     "the optional Companies House download reads a key from the CH_API_KEY environment variable.",
     size=9, colour=MUTED)
for title, rel in CODE_FILES:
    doc.add_heading(f"{title}  ({rel})", level=2)
    text = (ROOT / rel).read_text(encoding="utf-8").rstrip("\n")
    for line in text.split("\n"):
        p = doc.add_paragraph()
        pf = p.paragraph_format
        pf.space_after = pf.space_before = Pt(0)
        pf.line_spacing = 1.0
        pf.left_indent = Cm(0.2)
        run = p.add_run(line if line else " ")
        run.font.name = "Courier New"
        run.element.rPr.rFonts.set(qn("w:eastAsia"), "Courier New")
        run.font.size = Pt(6.5)
        run.font.color.rgb = RGBColor(0x22, 0x22, 0x22)

# page numbers in the footer
footer = doc.sections[0].footer.paragraphs[0]
footer.alignment = WD_ALIGN_PARAGRAPH.RIGHT
for kind, text in (("begin", None), (None, "PAGE"), ("end", None)):
    run = footer.add_run()
    run.font.size, run.font.color.rgb = Pt(8), MUTED
    if kind:
        fld = OxmlElement("w:fldChar")
        fld.set(qn("w:fldCharType"), kind)
        run._r.append(fld)
    else:
        instr = OxmlElement("w:instrText")
        instr.set(qn("xml:space"), "preserve")
        instr.text = text
        run._r.append(instr)
lead = footer.runs[0]._r
pre = footer.add_run("UK Company Financial Health Review · page ")
pre.font.size, pre.font.color.rgb = Pt(8), MUTED
lead.addprevious(pre._r)

OUT.parent.mkdir(exist_ok=True)
doc.save(OUT)
print(f"Saved {OUT}")

# PDF copy (needs LibreOffice; skipped quietly if it is not installed)
try:
    subprocess.run(["soffice", "--headless", "--convert-to", "pdf", "--outdir", str(OUT.parent), str(OUT)],
                   check=True, capture_output=True, timeout=180)
    print(f"Saved {OUT.with_suffix('.pdf')}")
except (FileNotFoundError, subprocess.CalledProcessError, subprocess.TimeoutExpired):
    print("LibreOffice not found: open the .docx and use File > Save as PDF.")
```

A7. Python: shared settings (GitHub address) (src/config.py)

```
"""
Project settings shared by the build scripts.

REPO_URL is the GitHub repository this project is published in. Links in the
Excel workbook and the report point here, so they keep working after someone
downloads a single file. Set the REPO_URL environment variable to override it
(for example, after forking the repository).
"""

import os

REPO_URL = os.environ.get(
    "REPO_URL", "https://github.com/Mumeen-Gbolagade/UK-Financial-Firms-Health-Review-"
).rstrip("/")
BRANCH = os.environ.get("REPO_BRANCH", "main")


def github_link(path: str) -> str:
    """GitHub page for a file or folder in this repository, e.g. 'report/x.pdf'."""
    path = path.strip("/")
    kind = "blob" if "." in path.rsplit("/", 1)[-1] else "tree"
    return f"{REPO_URL}/{kind}/{BRANCH}/{path}"
```

A8. Python: link checker (src/check_links.py)

```
"""
```

```
Check every link in the project before uploading to GitHub.

  - README.md: relative file links, image paths and #section anchors
  - Excel workbook: links between sheets, and links to files in this repository
  - Word report: links to files in this repository
  - With --online: also requests each external web address

Run from the project folder:
    python src/check_links.py            # offline checks
    python src/check_links.py --online   # also test external URLs (needs requests)
"""

from __future__ import annotations

import re
import sys
import zipfile
from pathlib import Path

from openpyxl import load_workbook

sys.path.insert(0, str(Path(__file__).resolve().parent))
from config import BRANCH, REPO_URL  # noqa: E402

ROOT = Path(__file__).resolve().parents[1]
problems: list[str] = []
external: set[str] = set()


def github_slug(heading: str) -> str:
    s = re.sub(r"[^\w\s-]", "", heading.strip().lower())
    return re.sub(r"\s", "-", s)


def repo_path(url: str) -> Path | None:
    """Map a link to this repository back to a local path."""
    for kind in ("blob", "tree", "raw"):
        prefix = f"{REPO_URL}/{kind}/{BRANCH}/"
        if url.startswith(prefix):
            return ROOT / url[len(prefix):]
    return None


def check_url(url: str, where: str) -> None:
    local = repo_path(url)
    if local is not None:
        if not local.exists():
            problems.append(f"{where}: {url} -> missing file {local.relative_to(ROOT)}")
    elif url.startswith("http"):
        external.add(url)


# ---- README.md
readme = (ROOT / "README.md").read_text(encoding="utf-8")
body = re.sub(r"```.*?```", "", readme, flags=re.S)          # ignore code blocks
anchors = {github_slug(h) for h in re.findall(r"^#{1,6} (.+)$", body, re.M)}
links = re.findall(r"!?\[[^\]]*\]\(([^)\s]+)\)", body)
for target in links:
    if target.startswith("#"):
        if target[1:] not in anchors:
            problems.append(f"README.md: anchor {target} has no matching heading")
    elif target.startswith("http"):
        check_url(target, "README.md")
    elif not (ROOT / target).exists():
        problems.append(f"README.md: {target} does not exist")
print(f"README.md: {len(links)} links checked")

# ---- Excel workbook
wb = load_workbook(ROOT / "outputs" / "UK_Financial_Health_Review.xlsx")
n = 0
for ws in wb.worksheets:
    for row in ws.iter_rows():
        for cell in row:
            h = cell.hyperlink
            if not h:
                continue
            n += 1
            where = f"Workbook {ws.title}!{cell.coordinate}"
            if h.location:
                if h.location.split("!")[0].strip("'") not in wb.sheetnames:
                    problems.append(f"{where}: sheet in {h.location} does not exist")
            elif h.target and h.target.startswith("http"):
                check_url(h.target, where)
            else:
                problems.append(f"{where}: relative link {h.target} breaks once the file is downloaded")
print(f"Workbook: {n} links checked")

# ---- Word report
docx = ROOT / "report" / "UK_Financial_Health_Review_Report.docx"
with zipfile.ZipFile(docx) as z:
    rels = z.read("word/_rels/document.xml.rels").decode("utf-8")
urls = re.findall(r'Target="(http[^"]+)"[^>]*TargetMode="External"', rels)
for u in urls:
    check_url(u.replace("&amp;", "&"), "Report")
print(f"Report: {len(urls)} links checked")

# ---- optional online check
if "--online" in sys.argv:
    import requests
```

```
    for url in sorted(external):
        try:
            r = requests.get(url, timeout=20, allow_redirects=True, headers={"User-Agent": "link-check"})
            if r.status_code >= 400:
                problems.append(f"HTTP {r.status_code}: {url}")
        except requests.RequestException as exc:
            problems.append(f"Unreachable: {url} ({exc.__class__.__name__})")
    print(f"Online: {len(external)} external addresses requested")
else:
    print(f"{len(external)} external addresses not requested (add --online to test them)")

if problems:
    print("\nProblems found:")
    for p in problems:
        print("  -", p)
    sys.exit(1)
print("\nAll links OK.")
```